**YORK UNIVERSITY**

Lassonde School of Engineering

**EECS 3311 - Software Design**

Fall 2026

# STAGE 1 PROJECT DESIGN REPORT

## Adaptive AI Study & Learning Agent

**Prepared by**



| Name | Email | Student Number |
|---|---|---|
| Armita Shahim | shahim@my.yorku.ca | 221962238 |



Instructor: Song Wang

Section: A

Submission Date: October 9, 2026

# Table of Contents


Task 1:
- 1.1 Project Description
  - Project Purpose and Problem
  - Target Users
  - Agent Functionality
  - Why an AI Agent Is Appropriate
  - AI Model and Integration
  - Overall Architecture and Interfaces
- 1.2 Feature Specification
  - F01 - Course and Academic Task Management
  - F02 - Weekly Availability and Commitment Management
  - F03 - Syllabus Import and Deadline Extraction
  - F04 - Weekly Study Plan Generation
  - F05 - Study Plan Revision
  - F06 - Study Session and Task Progress Recording
  - F07 - Adaptive Task Duration Estimation
  - F08 - Course Material Import and Retrieval
  - F09 - Practice Quiz Generation and Assessment
  - F10 - Weak-Topic Review Recommendations

Task 2:
- 2.1.1 Class Diagram
- 2.1.2 Design Pattern Explanations
  - 1. Strategy - Task Duration Estimation
  - 2. Adapter - AI Model Integration
  - 3. Command - Agent Tool Execution
  - 4. Facade - Unified Application Operations
  - 5. MVC - User Interface and Application Logic
- 2.2.1 Use-Case Diagram
- 2.2.2 Detailed Use-Case Descriptions
  - UC01 - Manage Courses and Tasks
  - UC02 - Manage Weekly Availability
  - UC03 - Import Syllabus Tasks
  - UC04 - Generate Weekly Study Plan
  - UC05 - Revise Study Plan
  - UC06 - Record Study Progress
  - UC07 - Adjust Task Duration Estimate
  - UC08 - Import and Retrieve Course Materials
  - UC09 - Take and Review Practice Quiz
  - UC10 - Get and Accept Review Recommendations

- 2.3 Sequence Diagrams
  - SD01 - Create & Remove an Academic Task
  - SD02 - Save Weekly Availability or Commitment
  - SD03 - Import & Confirm Syllabus Tasks
  - SD04A - Generate or Revise a Study Plan
  - SD05 - Record Study Progress
  - SD06 - Adjust Task Duration Estimate
  - SD07 - Import Course Material
  - SD08A - Generate Practice
  - SD09A - Generate Review Recommendations
  - SD10 - Mark Study Plan as Needing Revision
- Task 3:
  - Feature-to-Design Traceability
- Task 4:
  - Explain How Each Feature Is Realized
    - F01 - Course and Academic Task Management
    - F02 - Weekly Availability and Commitment Management
    - F03 - Syllabus Import and Deadline Extraction
    - F04 - Weekly Study Plan Generation
    - F05 - Study Plan Revision
    - F06 - Study Session and Task Progress Recording
    - F07 - Adaptive Task Duration Estimation
    - F08 - Course Material Import and Retrieval
    - F09 - Practice Quiz Generation and Assessment
    - F10 - Weak-Topic Review Recommendations

# Task 1:

## 1.1 Project Description

### Project Purpose and Problem

The Adaptive AI Study & Learning Agent is designed to help university and college students manage their academic workload and improve their learning. Students often struggle to balance multiple courses, deadlines, and personal commitments like part-time jobs. They may underestimate task durations, create unrealistic study schedules, or have difficulty identifying which topics need additional attention.

The proposed system addresses these problems by creating personalized study plans and adapting them using the student's recorded study behaviour and performance. It combines scheduling, learning support, and progress tracking to help students make more informed decisions about how to use their study time.

### Target Users

The primary users are university and college students who need support organizing their coursework around classes, employment, and other commitments. The system is particularly useful for students whose availability, workload, and learning needs change over time.

### Agent Functionality

Students can enter their courses, availability, personal commitments, academic deadlines, and estimated task durations. They can also upload text-based PDF course syllabi, from which the system extracts assignment deadlines and exam dates for review and confirmation. The agent uses the confirmed information to generate personalized study plans around existing commitments.

When a student misses a study session, changes their availability, or takes longer than expected to complete a task, the agent can propose revisions to the remaining study plan. Students can review and accept these changes. If the available time is insufficient, the system identifies the scheduling problem and suggests adjustments.

Students can upload text-based PDF course materials and select a topic to generate a multiple-choice practice quiz. After submission, the system provides a score and answer explanations and saves the results by course and topic. These results help identify topics requiring further practice. The agent recommends focused review tasks that students can confirm and include in future study plans.

The system maintains a persistent history of completed study sessions, estimated and actual task durations, quiz results, and previous plans. It uses this history to improve future time

estimates and study recommendations. For example, if similar tasks consistently take longer than expected, the system allocates more realistic amounts of time in future plans.

## Why an AI Agent Is Appropriate

This problem requires interpreting changing information and coordinating several actions. The agent must consider deadlines, availability, previous performance, and relevant course materials when deciding what to recommend.

An AI agent is appropriate because it can combine reasoning, planning, retrieval, memory, and tool use. For example, after a student completes a quiz, the agent can examine their performance, retrieve material related to difficult topics, recommend additional practice, and propose a revised study schedule.

## AI Model and Integration

The system will use DeepSeek-V4.1-Flash through the DeepSeek API, using the model identifier `deepseek-flash`. The model will extract academic information from course syllabi, generate practice quizzes and answer explanations from selected materials, recommend focused review activities, and support study-planning decisions using information supplied by the application.

An agent coordination component will manage interactions between the model and tools for retrieving course materials, accessing student history, and generating or modifying study plans. Deterministic software components will validate inputs and proposed schedules, check for conflicts, and store application data. Students will be able to review proposed plan changes before accepting them.

## Overall Architecture and Interfaces

The system will provide a graphical user interface for managing courses, academic tasks, weekly availability, study plans, course materials, quizzes, and review recommendations. Basic progress and history will be displayed within the task, session, and quiz screens. A command-line interface will also provide access to the major agent functions.

Both interfaces will use the same application services and domain logic. These services will coordinate the planning and learning components, communicate with the AI model, and access persistent storage. This structure will keep behaviour consistent across the GUI and CLI while allowing individual components to be developed and tested separately.

## 1.2 Feature Specification

The initial implementation will support one student profile with local persistent storage. Planning will cover one selected week at a time, using manually entered weekly availability and commitments. Uploads will be limited to text-based PDFs within configured file-size and text-length limits.

### F01 - Course and Academic Task Management

**Description:** Organizes courses and academic tasks, including assignments, exam preparation, and review activities.

- **User Interaction:** The student opens a course screen to add, edit, or remove courses and tasks.
- **Input:** Course name; task name, type, deadline, priority, and estimated study duration.
- **Output:** Saved courses and tasks, with their completion status.
- **AI Involvement:** Deterministic.
- **Expected Workflow:** The system validates and saves the information, then updates the displayed workload. Changes to scheduled tasks mark the current plan as needing revision. Course deletion is permitted only when no tasks, including archived tasks, or course materials remain associated with the course.
- **Error/Alternative Cases:** Missing required fields, invalid dates, and non-positive duration estimates are rejected with an explanation. Task removal requires confirmation and removes associated future sessions while retaining completed-session history.

### F02 - Weekly Availability and Commitment Management

**Description:** Records recurring weekly study availability and commitments, such as classes and employment.

- **User Interaction:** The student adds, edits, or removes periods in a weekly schedule editor.
- **Input:** Day of the week, start time, end time, and period type: availability or commitment.
- **Output:** Saved weekly periods used to calculate available study time.
- **AI Involvement:** Deterministic.
- **Expected Workflow:** The system validates and stores each period. Scheduling tools exclude commitment periods from study availability. Changes mark an existing plan as needing revision.
- **Error/Alternative Cases:** Invalid time ranges and overlapping availability entries are rejected. Commitments take precedence where they overlap availability. If no usable study time remains, the system informs the student.

### F03 - Syllabus Import and Deadline Extraction

**Description:** Extracts proposed assignment deadlines and exam dates from a text-based PDF syllabus.

- **User Interaction:** The student uploads a syllabus for a course, reviews and corrects the extracted items, and confirms which items to save.
- **Input:** A text-based PDF syllabus and its course association.
- **Output:** Confirmed academic tasks and deadlines.
- **AI Involvement:** Hybrid.
- **Expected Workflow:** The system extracts PDF text and asks the model to return structured task and date information. It validates the response and displays candidate items. The student corrects dates and supplies missing task details, including duration estimates, before saving.
- **Error/Alternative Cases:** Unsupported, oversized, unreadable, or scanned PDFs produce an explanation and a manual-entry option. Ambiguous dates require correction. Invalid model output or API failure leaves existing tasks unchanged.

### F04 - Weekly Study Plan Generation

**Description:** Proposes a weekly study schedule based on deadlines, priorities, available time, remaining task effort, and accepted duration estimates.

- **User Interaction:** The student selects a week, chooses **Generate Plan**, reviews the proposal, and accepts or rejects it.
- **Input:** Saved tasks, availability, commitments, selected week, and relevant study history.
- **Output:** Proposed study sessions linked to tasks, with a brief priority explanation and any unscheduled work.
- **AI Involvement:** Hybrid.
- **Expected Workflow:** The agent retrieves tasks and history through application tools, recommends an ordering of tasks, and invokes a deterministic scheduling tool. The system checks that sessions fit available periods, avoid overlaps, and occur before task deadlines. Acceptance saves the plan.
- **Error/Alternative Cases:** Missing availability or duration estimates prevent generation. Insufficient time produces a partial plan with unscheduled effort clearly identified. Invalid model output or API failure prevents a new plan from being saved.

### F05 - Study Plan Revision

**Description:** Reorganizes future sessions in the selected week after workload, availability, or recorded progress changes.

- **User Interaction:** The student selects **Revise Plan**, reviews the proposed changes, and accepts or rejects them.

- **Input:** The current plan, updated tasks and availability, recorded session outcomes, and remaining-effort estimates.
- **Output:** A revised proposal with a brief explanation of significant changes.
- **AI Involvement:** Hybrid.
- **Expected Workflow:** The agent retrieves unfinished work and current constraints, reassesses priorities, and invokes the scheduling tool to rebuild future sessions. The system validates the proposal. Acceptance replaces future sessions while preserving past session records.
- **Error/Alternative Cases:** Insufficient time is reported as unscheduled work. Rejection preserves the current plan and any conflict warning. Invalid model output or API failure leaves the existing plan unchanged.

### F06 - Study Session and Task Progress Recording

**Description:** Records session outcomes and actual study time, and updates remaining task effort.

- **User Interaction:** The student opens a scheduled session and records whether it was completed, partially completed, or missed.
- **Input:** Session outcome, actual minutes studied, task completion status, and estimated remaining effort for unfinished tasks.
- **Output:** Saved session history and updated task progress.
- **AI Involvement:** Deterministic.
- **Expected Workflow:** The system validates and saves the record, totals actual study time for the task, and updates its remaining effort. Outcomes affecting future sessions mark the plan as needing revision. A task is marked finished only when the student confirms completion.
- **Error/Alternative Cases:** Negative durations are rejected, and missed sessions record zero study time. Unfinished tasks require a remaining-effort estimate. Editing a record updates it rather than adding duplicate study time.

### F07 - Adaptive Task Duration Estimation

**Description:** Suggests more realistic task durations using previous estimated and actual completion times.

- **User Interaction:** When creating or editing an unstarted task, the student requests an adjusted estimate and accepts or overrides it.
- **Input:** The student's initial estimate and completed tasks of the same task type.
- **Output:** A suggested duration and the number of historical tasks supporting it.
- **AI Involvement:** Deterministic.
- **Expected Workflow:** The system calculates the average actual-to-original-estimated duration ratio from comparable completed tasks and applies it to the new estimate. Original estimates are retained separately from adjusted estimates. Accepted estimates become available to the planning agent.

- **Error/Alternative Cases:** With fewer than three comparable completed tasks, the original estimate is retained. Records with zero or missing durations are excluded. Suggestions never overwrite an estimate without student acceptance.

### F08 - Course Material Import and Retrieval

**Description:** Stores text-based PDF course materials and makes their extracted content available to quiz and review tools.

- **User Interaction:** The student uploads a PDF for a selected course, enters a title, and later selects the material for a learning activity.
- **Input:** A text-based PDF, material title, and course association.
- **Output:** A saved material record containing extracted text.
- **AI Involvement:** Deterministic.
- **Expected Workflow:** The system validates file type, size, and extracted text length, then stores the content locally. Retrieval tools load material by its saved identifier. The prototype uses explicit material selection rather than semantic search.
- **Error/Alternative Cases:** Unsupported, oversized, corrupted, password-protected, scanned, or text-empty files are rejected with an explanation. Materials exceeding the text limit require a shorter PDF.

### F09 - Practice Quiz Generation and Assessment

**Description:** Generates a short multiple-choice quiz from selected material, scores submitted answers, and saves feedback.

- **User Interaction:** The student selects material, enters one topic label, requests five questions, answers them, and submits the attempt.
- **Input:** Selected material, a topic label, and the student's answers.
- **Output:** A saved five-question quiz, score, answer explanations, and attempt record associated with the course and topic.
- **AI Involvement:** Hybrid: AI generates questions, the answer key, and explanations; deterministic code scores answers.
- **Expected Workflow:** The model generates questions about the chosen topic using the material. The system checks that each question has four choices, one designated correct answer, and an explanation. After submission, it calculates and saves the score and displays feedback. Reusing a saved topic label allows results to be grouped consistently.
- **Error/Alternative Cases:** Missing or insufficient relevant content, malformed output, or API failure prevents quiz creation. Unanswered questions prompt confirmation and receive no credit if submitted. Each attempt is submitted once. Structural validation does not guarantee factual accuracy.

**F10 - Weak-Topic Review Recommendations**

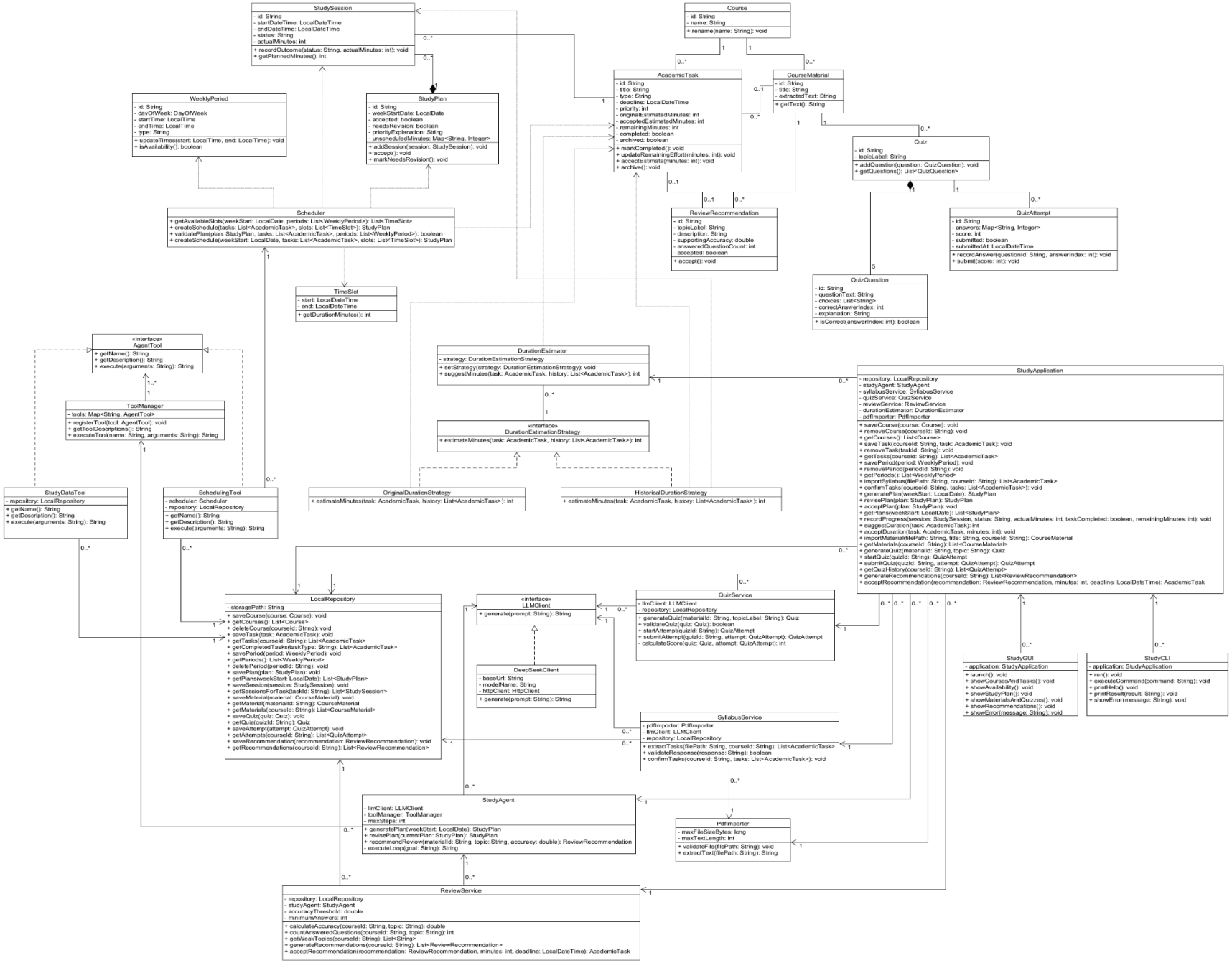
**Description:** Uses saved quiz results to identify topics needing practice and proposes review tasks for study planning.

- **User Interaction:** The student opens a course's recommendations, reviews the supporting scores, and accepts a suggested review task after setting its duration and deadline.
- **Input:** Quiz results grouped by course and saved topic label, associated materials, and upcoming academic deadlines.
- **Output:** Topic accuracy and proposed review tasks linked to relevant materials.
- **AI Involvement:** Hybrid.
- **Expected Workflow:** Deterministic code calculates topic accuracy. Topics below 60% accuracy across at least five answered questions are flagged. The agent retrieves the associated material through a tool and recommends a focused review activity. Acceptance creates a review task through F01, which can be scheduled through F04 or F05.
- **Error/Alternative Cases:** Topics with fewer than five answered questions are labelled as having insufficient evidence. No quiz history prompts the student to complete a quiz. API failure preserves calculated scores without creating a recommendation. Review tasks that cannot fit are reported as unscheduled during planning.

# Task 2:

## 2.1.1 Class Diagram

In the following page.


StudySession
- id: String
- startDateTime: LocalDateTime
- endDateTime: LocalDateTime
- status: String
- actualMinutes: int
+ recordOutcome(status: String, actualMinutes: int): void
+ getPlannedMinutes(): int
Course
- id: String
- name: String
+ rename(name: String): void
WeeklyPeriod
- id: String
- dayOfWeek: DayOfWeek
- startTime: LocalTime
- endTime: LocalTime
- type: String
+ updateTimes(start: LocalTime, end: LocalTime): void
+ isAvailability(): boolean
StudyPlan
- id: String
- weekStartDate: LocalDate
- accepted: boolean
- needsRevision: boolean
- priorityExplanation: String
- unscheduledMinutes: Map<String, Integer>
+ addSession(session: StudySession): void
+ accept(): void
+ markNeedsRevision(): void
AcademicTask
- id: String
- title: String
- type: String
- deadline: LocalDateTime
- priority: int
- originalEstimatedMinutes: int
- acceptedEstimatedMinutes: int
- remainingMinutes: int
- completed: boolean
- archived: boolean
+ markCompleted(): void
+ updateRemainingEffort(minutes: int): void
+ acceptEstimate(minutes: int): void
+ archive(): void
CourseMaterial
- id: String
- title: String
- extractedText: String
+ getText(): String
Quiz
- id: String
- topicLabel: String
+ addQuestion(question: QuizQuestion): void
+ getQuestions(): List<QuizQuestion>
ReviewRecommendation
- id: String
- topicLabel: String
- description: String
- supportingAccuracy: double
- answeredQuestionCount: int
- accepted: boolean
+ accept(): void
QuizAttempt
- id: String
- answers: Map<String, Integer>
- score: int
- submitted: boolean
- submittedAt: LocalDateTime
+ recordAnswer(questionId: String, answerIndex: int): void
+ submit(score: int): void
QuizQuestion
- id: String
- questionText: String
- choices: List<String>
- correctAnswerIndex: int
- explanation: String
+ isCorrect(answerIndex: int): boolean
Scheduler
+ getAvailableSlots(weekStart: LocalDate, periods: List<WeeklyPeriod>): List<TimeSlot>
+ createSchedule(tasks: List<AcademicTask>, slots: List<TimeSlot>): StudyPlan
+ validatePlan(plan: StudyPlan, tasks: List<AcademicTask>, periods: List<WeeklyPeriod>): boolean
+ createSchedule(weekStart: LocalDate, tasks: List<AcademicTask>, slots: List<TimeSlot>): StudyPlan
TimeSlot
- start: LocalDateTime
- end: LocalDateTime
+ getDurationMinutes(): int
«interface»
AgentTool
+ getName(): String
+ getDescription(): String
+ execute(arguments: String): String
ToolManager
- tools: Map<String, AgentTool>
+ registerTool(tool: AgentTool): void
+ getToolDescriptions(): String
+ executeTool(name: String, arguments: String): String
DurationEstimator
- strategy: DurationEstimationStrategy
+ setStrategy(strategy: DurationEstimationStrategy): void
+ suggestMinutes(task: AcademicTask, history: List<AcademicTask>): int
«interface»
DurationEstimationStrategy
+ estimateMinutes(task: AcademicTask, history: List<AcademicTask>): int
OriginalDurationStrategy
+ estimateMinutes(task: AcademicTask, history: List<AcademicTask>): int
HistoricalDurationStrategy
+ estimateMinutes(task: AcademicTask, history: List<AcademicTask>): int
StudyApplication
- repository: LocalRepository
- studyAgent: StudyAgent
- syllabusService: SyllabusService
- quizService: QuizService
- reviewService: ReviewService
- durationEstimator: DurationEstimator
- pdfImporter: PdfImporter
+ saveCourse(course: Course): void
+ removeCourse(courseId: String): void
+ getCourses(): List<Course>
+ saveTask(courseId: String, task: AcademicTask): void
+ removeTask(taskId: String): void
+ getTasks(courseId: String): List<AcademicTask>
+ savePeriod(period: WeeklyPeriod): void
+ removePeriod(periodId: String): void
+ getPeriods(): List<WeeklyPeriod>
+ importSyllabus(filePath: String, courseId: String): List<AcademicTask>
+ confirmTasks(courseId: String, tasks: List<AcademicTask>): void
+ generatePlan(weekStart: LocalDate): StudyPlan
+ revisePlan(plan: StudyPlan): StudyPlan
+ acceptPlan(plan: StudyPlan): void
+ getPlans(weekStart: LocalDate): List<StudyPlan>
+ recordProgress(session: StudySession, status: String, actualMinutes: int, taskCompleted: boolean, remainingMinutes: int): void
+ suggestDuration(task: AcademicTask): int
+ acceptDuration(task: AcademicTask, minutes: int): void
+ importMaterial(filePath: String, title: String, courseId: String): CourseMaterial
+ getMaterials(courseId: String): List<CourseMaterial>
+ generateQuiz(materialId: String, topic: String): Quiz
+ startQuiz(quizId: String): QuizAttempt
+ submitQuiz(quizId: String, attempt: QuizAttempt): QuizAttempt
+ getQuizHistory(courseId: String): List<QuizAttempt>
+ generateRecommendations(courseId: String): List<ReviewRecommendation>
+ acceptRecommendation(recommendation: ReviewRecommendation, minutes: int, deadline: LocalDateTime): AcademicTask
StudyDataTool
- repository: LocalRepository
+ getName(): String
+ getDescription(): String
+ execute(arguments: String): String
SchedulingTool
- scheduler: Scheduler
- repository: LocalRepository
+ getName(): String
+ getDescription(): String
+ execute(arguments: String): String
LocalRepository
- storagePath: String
+ saveCourse(course: Course): void
+ getCourses(): List<Course>
+ deleteCourse(courseId: String): void
+ saveTask(task: AcademicTask): void
+ getTasks(courseId: String): List<AcademicTask>
+ getCompletedTasks(taskType: String): List<AcademicTask>
+ savePeriod(period: WeeklyPeriod): void
+ getPeriods(): List<WeeklyPeriod>
+ deletePeriod(periodId: String): void
+ savePlan(plan: StudyPlan): void
+ getPlans(weekStart: LocalDate): List<StudyPlan>
+ saveSession(session: StudySession): void
+ getSessionsForTask(taskId: String): List<StudySession>
+ saveMaterial(material: CourseMaterial): void
+ getMaterial(materialId: String): CourseMaterial
+ getMaterials(courseId: String): List<CourseMaterial>
+ saveQuiz(quiz: Quiz): void
+ getQuiz(quizId: String): Quiz
+ saveAttempt(attempt: QuizAttempt): void
+ getAttempts(courseId: String): List<QuizAttempt>
+ saveRecommendation(recommendation: ReviewRecommendation): void
+ getRecommendations(courseId: String): List<ReviewRecommendation>
«interface»
LLMClient
+ generate(prompt: String): String
DeepSeekClient
- baseUrl: String
- modelName: String
- httpClient: HttpClient
+ generate(prompt: String): String
QuizService
- llmClient: LLMClient
- repository: LocalRepository
+ generateQuiz(materialId: String, topicLabel: String): Quiz
+ validateQuiz(quiz: Quiz): boolean
+ startAttempt(quizId: String): QuizAttempt
+ submitAttempt(quizId: String, attempt: QuizAttempt): QuizAttempt
- calculateScore(quiz: Quiz, attempt: QuizAttempt): int
SyllabusService
- pdfImporter: PdfImporter
- llmClient: LLMClient
- repository: LocalRepository
+ extractTasks(filePath: String, courseId: String): List<AcademicTask>
+ validateResponse(response: String): boolean
+ confirmTasks(courseId: String, tasks: List<AcademicTask>): void
StudyGUI
- application: StudyApplication
+ launch(): void
+ showCoursesAndTasks(): void
+ showAvailability(): void
+ showStudyPlan(): void
+ showMaterialsAndQuizzes(): void
+ showRecommendations(): void
+ showError(message: String): void
StudyCLI
- application: StudyApplication
+ run(): void
+ executeCommand(command: String): void
+ printHelp(): void
+ printResult(result: String): void
+ showError(message: String): void
StudyAgent
- llmClient: LLMClient
- toolManager: ToolManager
- maxSteps: int
+ generatePlan(weekStart: LocalDate): StudyPlan
+ revisePlan(currentPlan: StudyPlan): StudyPlan
+ recommendReview(materialId: String, topic: String, accuracy: double): ReviewRecommendation
- executeLoop(goal: String): String
PdfImporter
- maxFileSizeBytes: long
- maxTextLength: int
+ validateFile(filePath: String): void
+ extractText(filePath: String): String
ReviewService
- repository: LocalRepository
- studyAgent: StudyAgent
- accuracyThreshold: double
- minimumAnswers: int
+ calculateAccuracy(courseId: String, topic: String): double
+ countAnsweredQuestions(courseId: String, topic: String): int
+ getWeakTopics(courseId: String): List<String>
+ generateRecommendations(courseId: String): List<ReviewRecommendation>
+ acceptRecommendation(recommendation: ReviewRecommendation, minutes: int, deadline: LocalDateTime): AcademicTask

## 2.1.2 Design Pattern Explanations

### 1. Strategy - Task Duration Estimation

**Problem addressed:** Task duration estimates can use either the student's original estimate or their completion history. The system needs to switch between these approaches.

**Participating classes and roles:** `DurationEstimationStrategy` defines the estimation operation. `OriginalDurationStrategy` returns the original estimate, while `HistoricalDurationStrategy` adjusts it using comparable completed tasks. `DurationEstimator` uses the selected strategy.

**Why it is appropriate:** Both approaches follow the same interface, so `DurationEstimator` can use either without changing how the rest of the application requests estimates. When there is insufficient history, the original estimate is used.

**Without the pattern:** Estimation rules would be combined inside `DurationEstimator`, making it harder to change one approach or add another.

### 2. Adapter - AI Model Integration

**Problem addressed:** The application needs to communicate with the DeepSeek API without spreading API-specific request and response handling throughout the system.

**Participating classes and roles:** `LLMClient` defines the interface expected by the application through `generate(prompt)`. `DeepSeekClient` implements this interface and acts as the adapter: it converts the prompt into a DeepSeek HTTP request and extracts the generated text from the response. The external DeepSeek API is the service being adapted.

**Why it is appropriate:** `StudyAgent`, `SyllabusService`, and `QuizService` depend on `LLMClient`. This keeps provider-specific details inside `DeepSeekClient` and allows another provider to be integrated through the same interface.

**Without the pattern:** Components using AI would need to handle provider-specific details themselves, making API changes or switching providers more difficult.

### 3. Command - Agent Tool Execution

**Problem addressed:** The agent needs to request different operations, such as retrieving student data and generating schedules, through a common execution interface.

**Participating classes and roles:** `AgentTool` is the command interface and defines `execute(arguments)`. `StudyDataTool` and `SchedulingTool` are concrete commands. `ToolManager` acts as the invoker: it finds a registered tool by name and executes it. `LocalRepository` and `Scheduler` are the receivers that perform the underlying work. `StudyAgent` requests tool execution through `ToolManager`.

**Why it is appropriate:** Each tool packages an operation behind the same interface. The agent can request an operation without knowing how data retrieval or scheduling is implemented. Additional tools can be registered without changing the tool execution process.

**Without the pattern:** The agent would need separate logic for calling each operation directly, making its reasoning loop more tightly connected to the repository and scheduling code.

## 4. Facade - Unified Application Operations

**Problem addressed:** The GUI and CLI need access to features that involve several services. Having each interface coordinate those services separately would duplicate workflow logic.

**Participating classes and roles:** `StudyApplication` acts as the facade, providing operations such as `generatePlan()`, `importSyllabus()`, and `submitQuiz()`. It coordinates `StudyAgent`, `SyllabusService`, `QuizService`, `ReviewService`, `DurationEstimator`, `PdfImporter`, and `LocalRepository`. `StudyGUI` and `StudyCLI` are clients of the facade.

**Why it is appropriate:** Both interfaces access the same application operations through one entry point. This keeps workflows consistent and lets the interfaces focus on collecting input and displaying results.

**Without the pattern:** The GUI and CLI would each need to know which services to call and in what order. Changes to a workflow could require updates in both interfaces.

## 5. MVC - User Interface and Application Logic

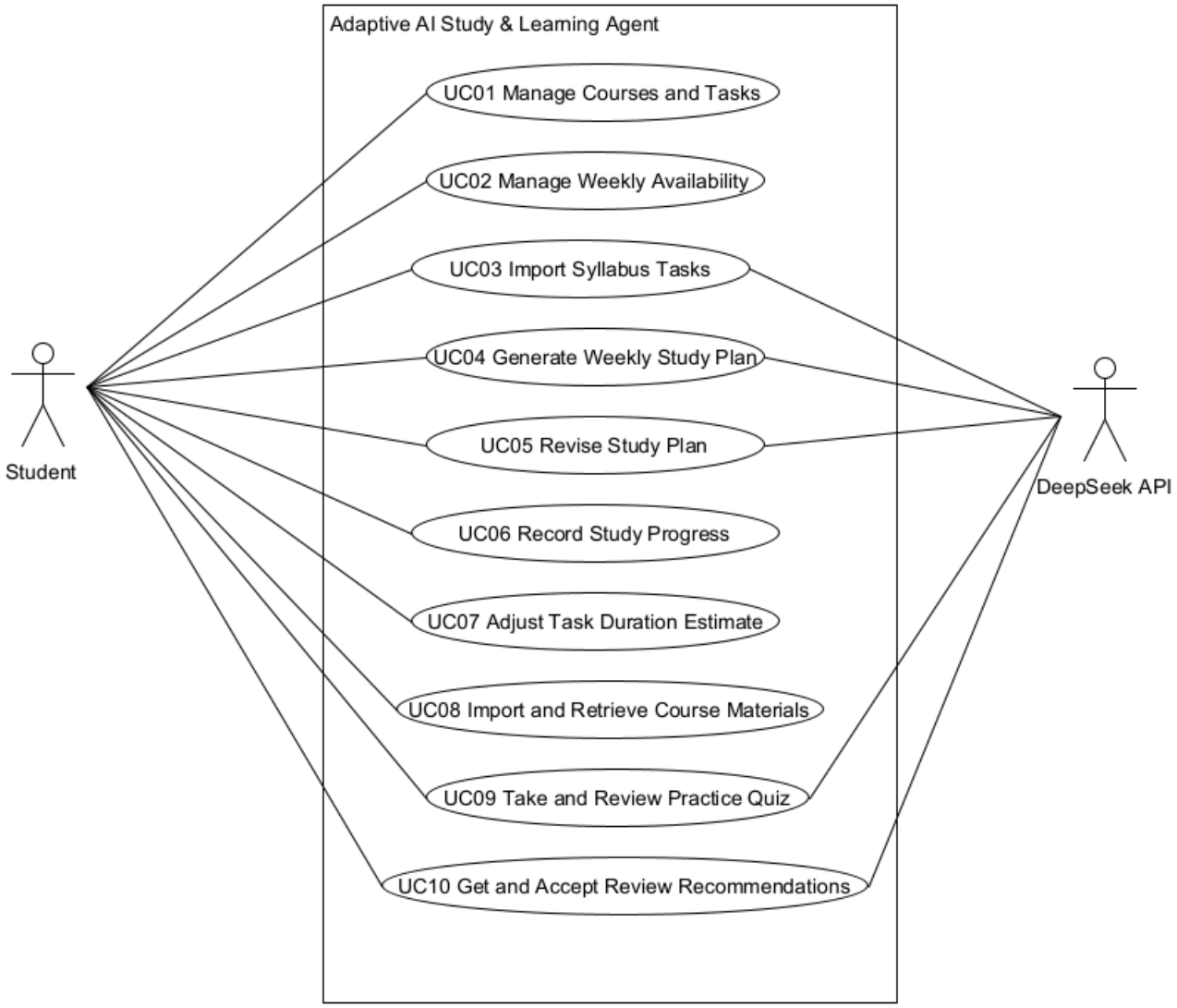
**Problem addressed:** The system provides both a GUI and a CLI. Presentation code needs to remain separate from study data and application logic so both interfaces can use the same functionality.

**Participating classes and roles:** `StudyGUI` and `StudyCLI` are the views: they collect input and display results. `StudyApplication` acts as the controller: it receives requests and coordinates the required operations. Domain classes such as `AcademicTask`, `StudyPlan`, and `Quiz`, together with the services and repository, form the model, which manages data and application behaviour.

**Why it is appropriate:** Both views use the same controller and model. This allows interface changes without rewriting scheduling, quiz assessment, or data storage logic. `StudyApplication` also serves as the facade; these roles complement each other.

**Without the pattern:** Application logic could become mixed into interface code, causing duplication between the GUI and CLI and making changes and testing more difficult.

## 2.2.1 Use-Case Diagram




## 2.2.2 Detailed Use-Case Descriptions

### UC01 - Manage Courses and Tasks

**Actor:** Student.

**Goal:** Organize courses and academic tasks used for study planning.

**Preconditions:** The application is running. A course must exist before a task can be added to it.

**Trigger:** The student opens the courses and tasks screen.

**Main Success Scenario:**

1. The system displays saved courses and their tasks.
2. The student creates or selects a course.
3. The student adds or edits a task, entering its title, type, deadline, priority, and estimated duration.
4. The system validates the information.
5. The system saves the changes and refreshes the displayed list. Changes affecting an accepted schedule mark it as needing revision.

**Alternative/Exception Flows:**

- Missing required information or invalid values: the system explains the problem and requests corrections.
- Task removal: after confirmation, the system archives the task and removes its future study sessions while preserving past records.
- Course removal: the system permits deletion only when no tasks, including archived tasks, or course materials remain linked to it.
- Saving fails: the system displays an error and does not report the changes as saved.

**Postconditions:** Successful changes are stored. Archived tasks are excluded from future planning, and existing study history is preserved.

**Related Feature:** F01 - Course and Academic Task Management.

## UC02 - Manage Weekly Availability

**Actor:** Student.

**Goal:** Define recurring study availability and fixed commitments used when generating study plans.

**Preconditions:** The application is running.

**Trigger:** The student opens the weekly availability screen.

**Main Success Scenario:**

1. The system displays saved availability periods and commitments.
2. The student adds or edits a period, specifying its day, start time, end time, and whether it represents availability or a commitment.
3. The system validates the times and checks for overlapping availability periods.
4. The system saves the period and refreshes the display.
5. Existing accepted plans affected by the change are marked as needing revision. Future planning excludes commitment times from available study windows.

**Alternative/Exception Flows:**

- Missing values or an end time that is not after the start time: the system requests corrections.
- Overlapping availability periods: the system asks the student to adjust or merge them.
- A commitment overlaps an availability period: the commitment takes precedence, and that time is excluded from scheduling.
- Period removal: the student confirms removal, and the system updates availability and marks affected plans as needing revision.
- Saving fails: the system displays an error without reporting success.

**Postconditions:** Successful changes are stored. Existing sessions remain unchanged until the student accepts a revised plan.

**Related Feature:** F02 - Weekly Availability and Commitment Management.

## UC03 - Import Syllabus Tasks

**Actors:** Student; DeepSeek API as a supporting actor.

**Goal:** Extract academic tasks from a syllabus and save them after student review.

**Preconditions:** A course exists, and the student has a text-based PDF syllabus.

**Trigger:** The student selects Import Syllabus for a course.

**Main Success Scenario:**

1. The student selects the syllabus PDF.
2. The system validates the file and extracts its text.
3. The system sends the text to the AI service to identify academic tasks and deadlines.
4. The system validates the response and displays draft tasks.
5. The student reviews the drafts, corrects details, removes unwanted entries, and supplies missing information, including task type, priority, and estimated duration.
6. The student confirms the selected tasks.
7. The system validates and saves the confirmed tasks under the selected course.

**Alternative/Exception Flows:**

- Unsupported, oversized, corrupted, password-protected, scanned, or text-empty PDF: the system explains the issue and requests a suitable file.
- Missing or ambiguous deadlines: the student must enter or correct them before saving.
- API failure or malformed response: the system displays an error and saves no tasks.
- No tasks identified: the system informs the student and allows manual task entry.
- Student cancels: draft tasks are discarded without being saved.
- Saving fails: the system reports the error without claiming success.

**Postconditions:** Confirmed tasks are stored and available for planning. Unconfirmed drafts are not stored.

**Related Feature:** F03 - AI-Assisted Syllabus Task Extraction.

## UC04 - Generate Weekly Study Plan

**Actors:** Student; DeepSeek API as a supporting actor.

**Goal:** Create and accept a weekly study plan that fits academic tasks around availability and commitments.

**Preconditions:** At least one unfinished, non-archived task exists with valid planning details. The student has configured weekly availability and commitments.

**Trigger:** The student selects a week and requests a study plan.

**Main Success Scenario:**

1. The system retrieves unfinished tasks, remaining effort, availability, commitments, and relevant study history.
2. The agent uses this information to propose task priorities and explain its decisions.
3. The agent invokes the scheduling tool to allocate study sessions to available slots.
4. The system validates the proposed plan against availability, commitments, task deadlines, and session overlaps. For the current week, past times are excluded.
5. The system displays the proposed sessions, priority explanation, and any unscheduled effort.
6. The student reviews and accepts the proposal.
7. The system saves the accepted plan and displays it.

**Alternative/Exception Flows:**

- Insufficient available time: the system displays a partial plan and identifies unscheduled effort. The student may accept the partial plan or adjust tasks or availability.
- No available slots: the system reports that no sessions can be scheduled for the selected week.
- Invalid schedule: the system requests a corrected proposal within the agent's step limit. If validation still fails, it reports the failure and saves no plan.
- API failure or agent step limit reached: the system explains that generation could not be completed.
- The student rejects the proposal: it is discarded, and existing saved plans remain unchanged.
- An accepted plan already exists for the week: the student is directed to UC05 to revise it.
- Saving fails: the system reports the error without claiming the plan was saved.

**Postconditions:** An accepted, valid plan is stored for the selected week. Task completion and recorded study time remain unchanged.

**Related Feature:** F04 - Personalized Weekly Study Plan Generation.

## UC05 - Revise Study Plan

**Actors:** Student; DeepSeek API as a supporting actor.

**Goal:** Update the future sessions of an accepted study plan while preserving past sessions and recorded progress.

**Preconditions:** An accepted plan exists for the selected week, and the week has not ended.

**Trigger:** The student requests a revision, possibly after changes to tasks, availability, or study progress.

**Main Success Scenario:**

1. The student selects the accepted plan and requests a revision.
2. The system retrieves the plan, updated tasks, remaining effort, availability, commitments, and recorded progress.
3. The agent identifies future sessions that can be replaced and preserves past sessions and recorded outcomes.
4. The agent uses the scheduling tool to propose new future sessions within the selected week.
5. The system validates the proposal against deadlines, availability, commitments, and session overlaps.
6. The system displays the proposed changes, their explanation, and any unscheduled effort.
7. The student accepts the revision.
8. The system replaces the plan's future sessions with the accepted proposal, preserves its history, and clears its revision flag.

**Alternative/Exception Flows:**

- Insufficient available time: the system identifies unscheduled effort and allows the student to accept a partial revision or adjust inputs.
- No future slots remain: the system explains that no additional sessions can fit in the selected week.
- The selected week has ended: revision is unavailable; the student can generate a plan for another week.
- Invalid proposal, API failure, or agent step limit reached: the system reports the failure and retains the existing plan.
- The student rejects the proposal: the existing plan remains unchanged.
- Saving fails: the existing saved plan is retained, and the system reports the error.

**Postconditions:** If accepted and saved, the revised plan becomes the active schedule for that week. Past sessions and recorded progress are preserved. Otherwise, the existing plan remains unchanged.

**Related Feature:** F05 - Adaptive Study Plan Revision.

## UC06 - Record Study Progress

**Actor:** Student.

**Goal:** Record a study session's outcome and update the associated task's completion status and remaining effort.

**Preconditions:** A saved study session exists, and its scheduled start time has passed.

**Trigger:** The student selects a session to record or edit its outcome.

**Main Success Scenario:**

1. The system displays the session, its associated task, and any previously recorded outcome.
2. The student selects completed, partially completed, or missed and enters actual study minutes.
3. The student indicates whether the academic task itself is finished. If unfinished, the student supplies an updated remaining-effort estimate.
4. The system validates the information.
5. The system saves the session outcome and updates the task. A finished task has zero remaining effort.
6. The system marks affected plans as needing revision and displays the updated progress.

**Alternative/Exception Flows:**

- Negative minutes, invalid status, or missing required information: the system requests corrections.
- Missed session: actual study minutes are recorded as zero.
- Session completed but task unfinished: the task remains open with the supplied remaining-effort estimate.
- Existing record edited: the previous outcome is replaced, and total actual study time is recalculated without double counting.
- Saving fails: the system reports the error without claiming the update succeeded.

**Postconditions:** The saved session outcome and task progress reflect the latest entry. Future sessions remain unchanged until a revision is accepted.

**Related Feature:** F06 - Study Progress and Session History Tracking.

## UC07 - Adjust Task Duration Estimate

**Actor:** Student.

**Goal:** Obtain a task duration suggestion based on completion history and choose whether to use it.

**Preconditions:** The student is creating or editing an unstarted task with a task type and a positive original duration estimate.

**Trigger:** The student requests an adjusted duration estimate.

**Main Success Scenario:**

1. The system retrieves completed tasks of the same task type.
2. The system excludes records with missing or zero original estimates or total actual study minutes.
3. With at least three eligible tasks, the system calculates the average actual-to-original-estimated duration ratio.
4. The system multiplies the new task's original estimate by that ratio.
5. The system displays the suggested duration and the number of supporting historical tasks.
6. The student accepts the suggestion or enters a different positive duration.
7. When the student saves the task, the system stores the chosen duration as its accepted estimate and remaining effort, while retaining the original estimate.

**Alternative/Exception Flows:**

- Fewer than three eligible completed tasks: the system suggests the original estimate and explains that there is insufficient history.
- Missing task type or invalid duration: the system requests corrections.
- The task has already started: the student updates the remaining effort through UC06 instead.
- Student dismisses the suggestion: the existing estimate remains unchanged.
- Saving fails: the system reports the error without claiming success.

**Postconditions:** If saved, the chosen estimate is available for planning. The original estimate remains available for historical comparisons.

**Related Feature:** F07 - Adaptive Task Duration Estimation.

## UC08 - Import and Retrieve Course Materials

**Actor:** Student.

**Goal:** Store course material and retrieve its extracted text for quizzes and review activities.

**Preconditions:** A course exists. Import requires a text-based PDF; retrieval requires a saved material record.

**Trigger:** The student opens a course's materials screen and selects Import Material or an existing material.

**Main Success Scenario:**

1. The system displays the course's saved materials.
2. The student selects a PDF and enters a material title.
3. The system validates the file type and size, extracts its text, and checks that the text is non-empty and within the length limit.
4. The system saves the title, extracted text, and course association.
5. The system refreshes the material list.
6. The student selects a saved material.
7. The system retrieves the material by its identifier and makes its text available for the selected learning activity.

**Alternative/Exception Flows:**

- Unsupported, oversized, corrupted, password-protected, scanned, or text-empty PDF: the system explains the issue and requests a suitable file.
- Extracted text exceeds the limit: the system requests a shorter PDF.
- Missing title: the system requests a title before saving.
- Selected material cannot be found: the system reports the issue and refreshes the list.
- Student cancels import: no material is saved.
- Saving or retrieval fails: the system reports the error without claiming success.

**Postconditions:** Successfully imported material is stored under the selected course. Retrieved text is available for use without modifying the saved material.

**Related Feature:** F08 - Course Material Import and Retrieval.

## UC09 - Take and Review Practice Quiz

**Actors:** Student; DeepSeek API as a supporting actor.

**Goal:** Practise a selected topic and receive a score and answer explanations.

**Preconditions:** Course material has been imported and contains text relevant to the selected topic.

**Trigger:** The student selects material, enters a topic label, and requests a practice quiz.

**Main Success Scenario:**

1. The system retrieves the selected material's text.

2. The system requests five multiple-choice questions from the AI service about the selected topic.
3. The system validates that there are exactly five questions, each with four choices, one designated correct answer, and an explanation.
4. The system saves the quiz, creates an attempt, and displays the questions without revealing the answers.
5. The student selects answers and submits the attempt.
6. The system scores the answers using the saved answer key and marks the attempt as submitted.
7. The system saves the attempt and displays the score, correct answers, and explanations.
8. The student can later view the saved results in the course's quiz history.

**Alternative/Exception Flows:**

- Missing topic or insufficient relevant material: the system requests a topic or different material.
- API failure or malformed output: the system reports the issue and creates no quiz.
- Unanswered questions: the system requests confirmation before submission. If confirmed, unanswered questions receive no credit.
- Attempt already submitted: further changes or duplicate submission are rejected.
- Student leaves before submission: no completed attempt is added to assessment history.
- Saving fails: the system reports the error without claiming the results were stored.

**Postconditions:** A successfully submitted attempt is stored once and linked to its quiz, course, and topic. Its answers are available for weak-topic analysis.

**Related Feature:** F09 - Practice Quiz Generation and Assessment.

## UC10 - Get and Accept Review Recommendations

**Actors:** Student; DeepSeek API as a supporting actor.

**Goal:** Identify weak topics and create focused review tasks from accepted recommendations.

**Preconditions:** A course exists with saved, submitted quiz attempts and associated course materials.

**Trigger:** The student opens the course's recommendations screen and requests recommendations.

**Main Success Scenario:**

1. The system retrieves submitted quiz attempts and groups results by course and topic label.

2. The system calculates each topic's accuracy using correctly answered questions divided by answered questions. Unanswered questions are excluded from this calculation.
3. Topics below 60% accuracy across at least five answered questions are flagged for review.
4. The agent retrieves associated course material through its data tool and uses the AI service to propose a focused review activity.
5. The system validates and saves the recommendation, including its supporting accuracy and answered-question count.
6. The student reviews the recommendation and enters a positive duration and a deadline.
7. The student accepts the recommendation.
8. The system creates a review task linked to the course and relevant material and marks the recommendation as accepted.

**Alternative/Exception Flows:**

- No submitted quiz history: the system prompts the student to complete a quiz.
- Fewer than five answered questions for a topic: the system labels it as having insufficient evidence.
- No topics meet the weak-topic criteria: the system reports that no review recommendations are currently needed.
- Relevant material is unavailable, or AI generation fails: calculated scores remain available, but no new recommendation is created.
- Invalid duration or deadline: the system requests corrections before acceptance.
- Student declines: no review task is created.
- Recommendation already accepted: the system prevents duplicate task creation.
- Saving fails: the system reports the error without claiming acceptance succeeded.

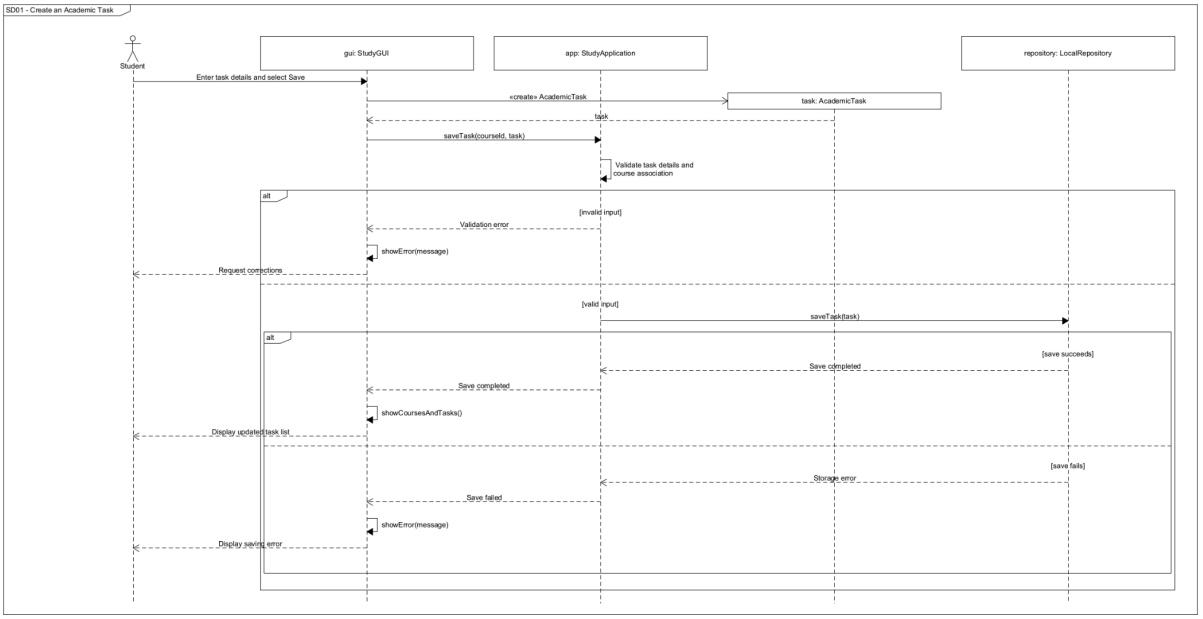
**Postconditions:** Successful acceptance creates one review task available for scheduling through UC04 or UC05. Existing schedules remain unchanged until a new or revised plan is accepted.

**Related Features:** F10 - Weak-Topic Review Recommendations, F01 - Course and Academic Task Management for creating the accepted review task.

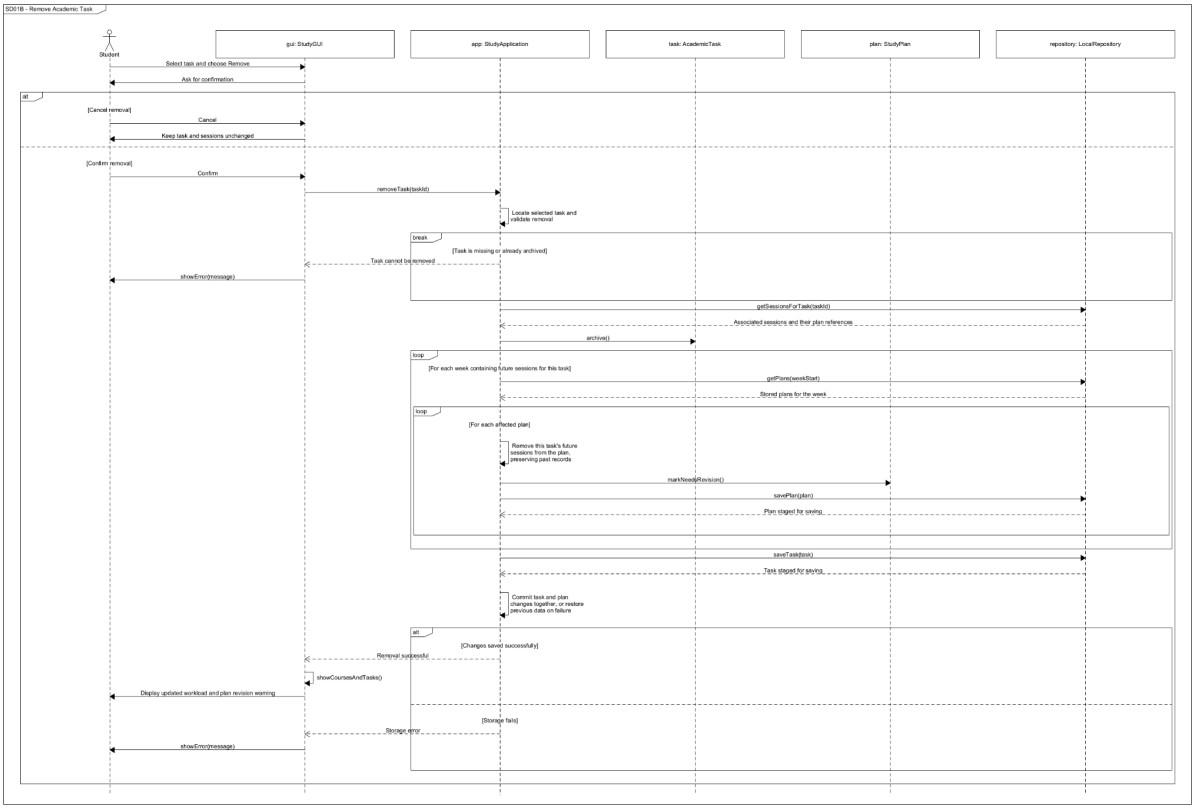
## 2.3 Sequence Diagrams

### SD01 - Create & Remove an Academic Task

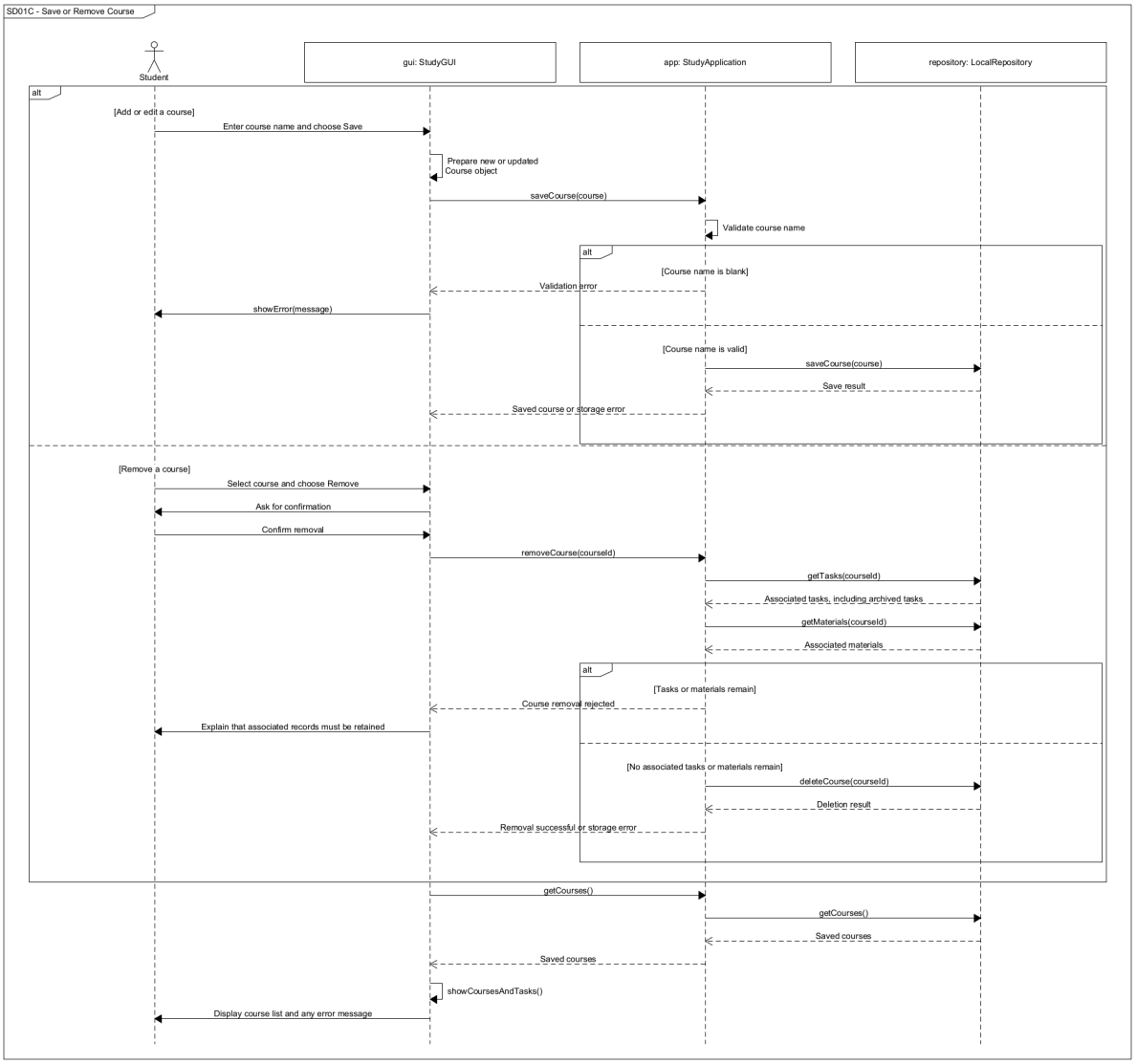
Create:



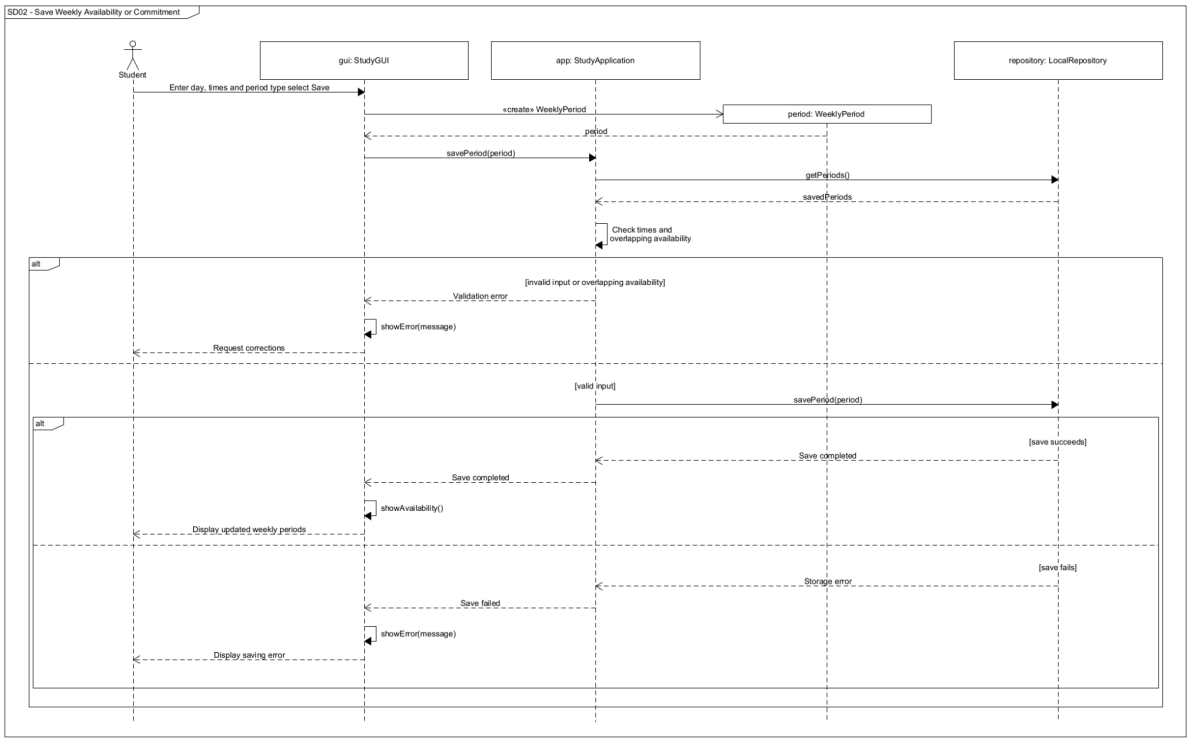

Remove:



Save or Remove a course:

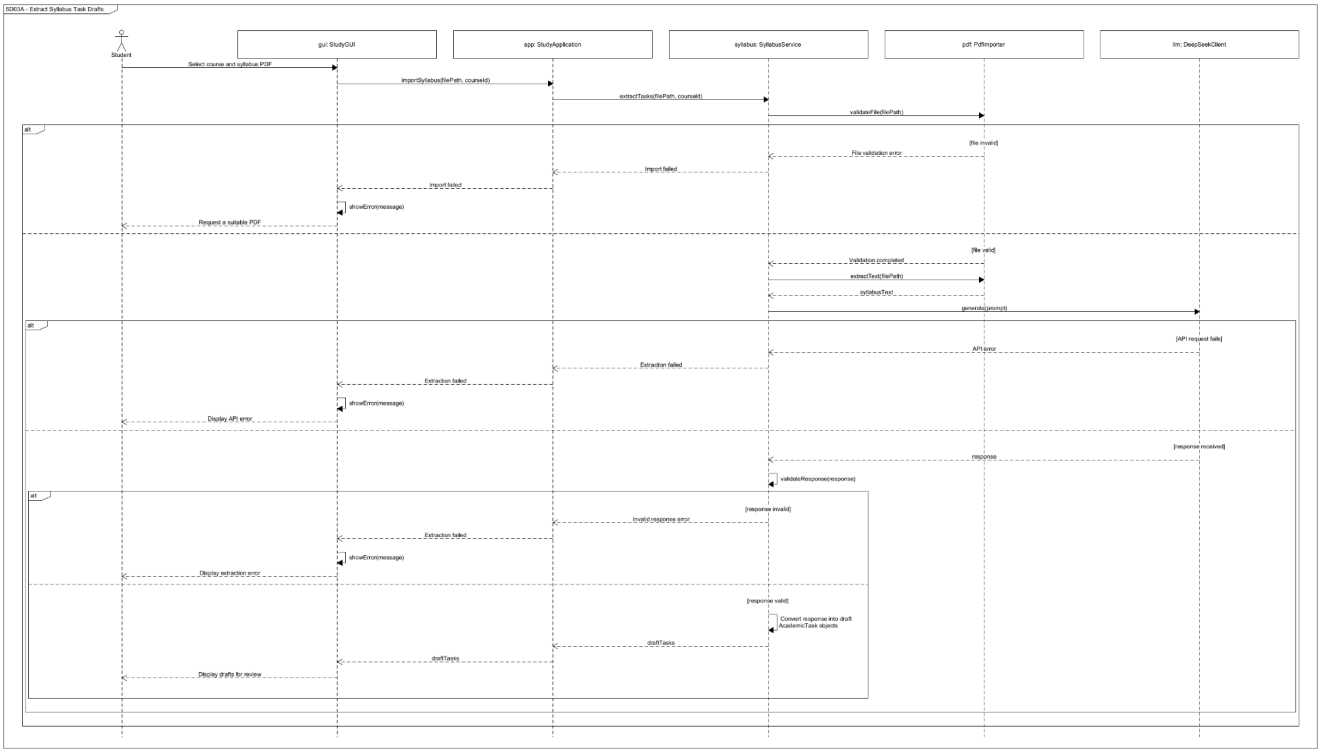


# SD02 - Save Weekly Availability or Commitment

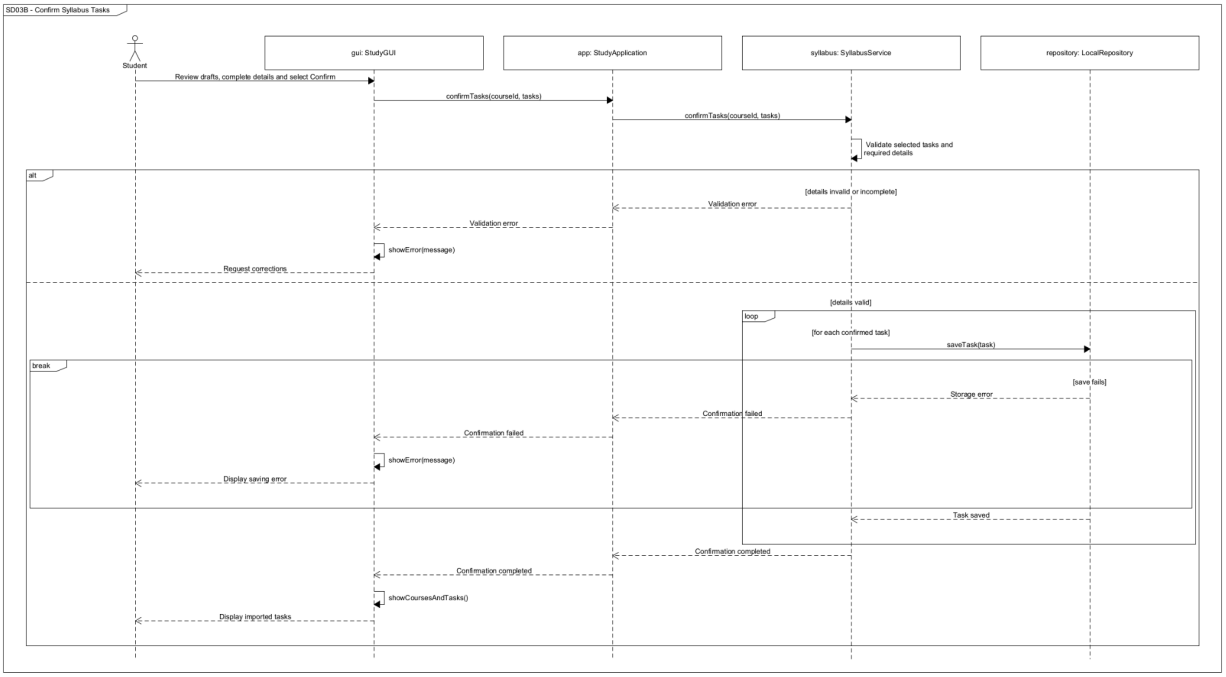



# SD03 - Import & Confirm Syllabus Tasks

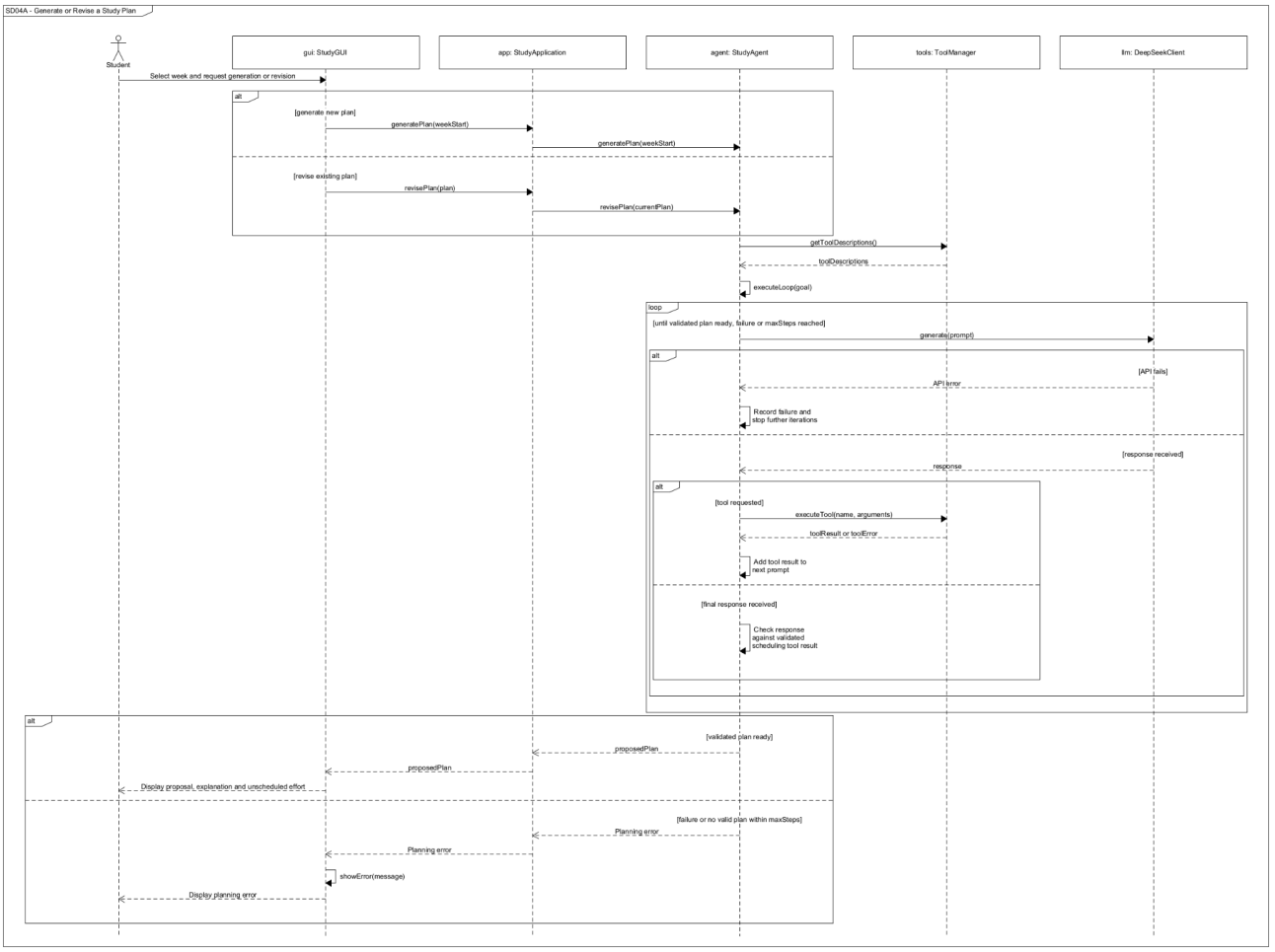
Import:

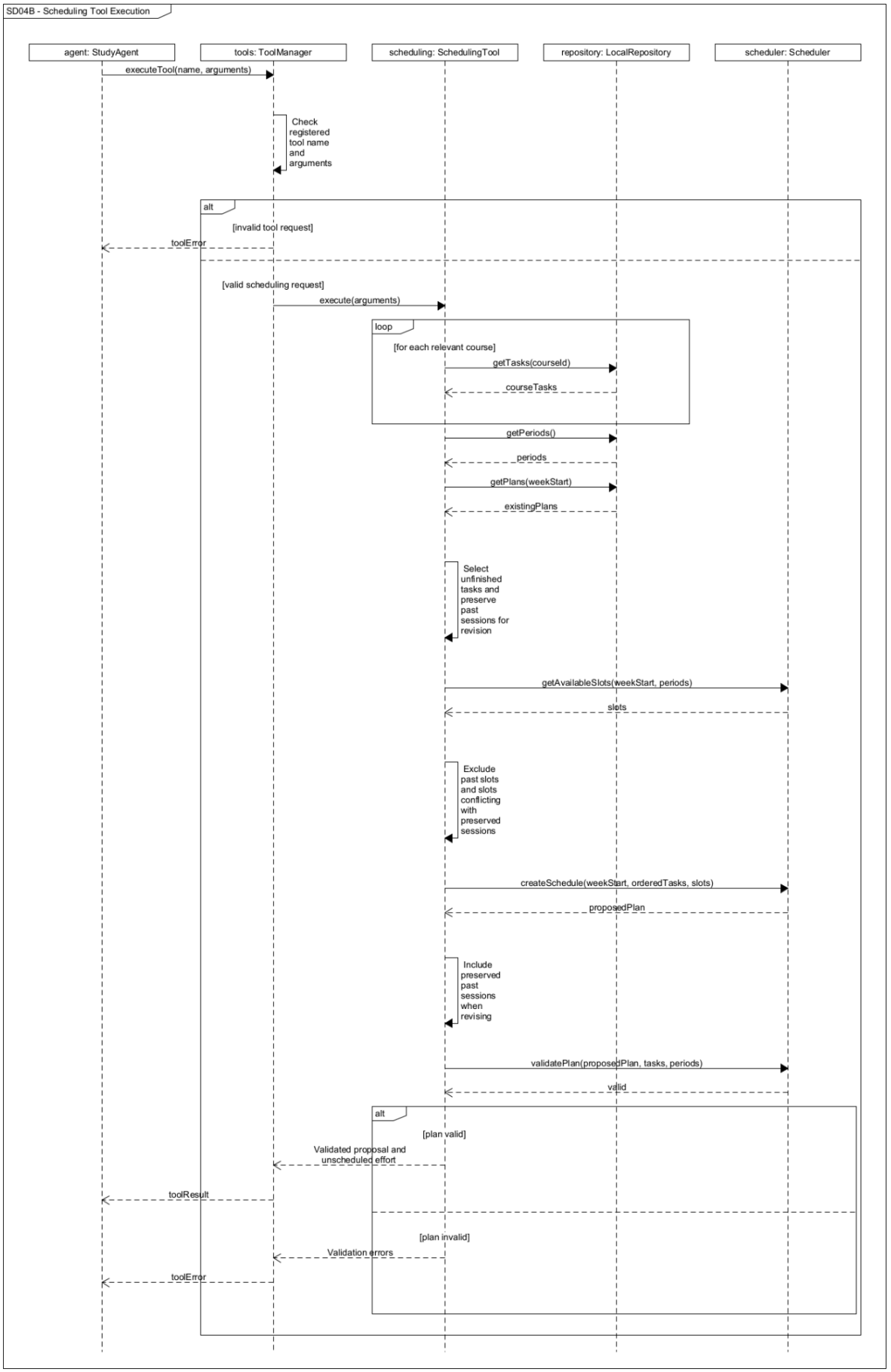


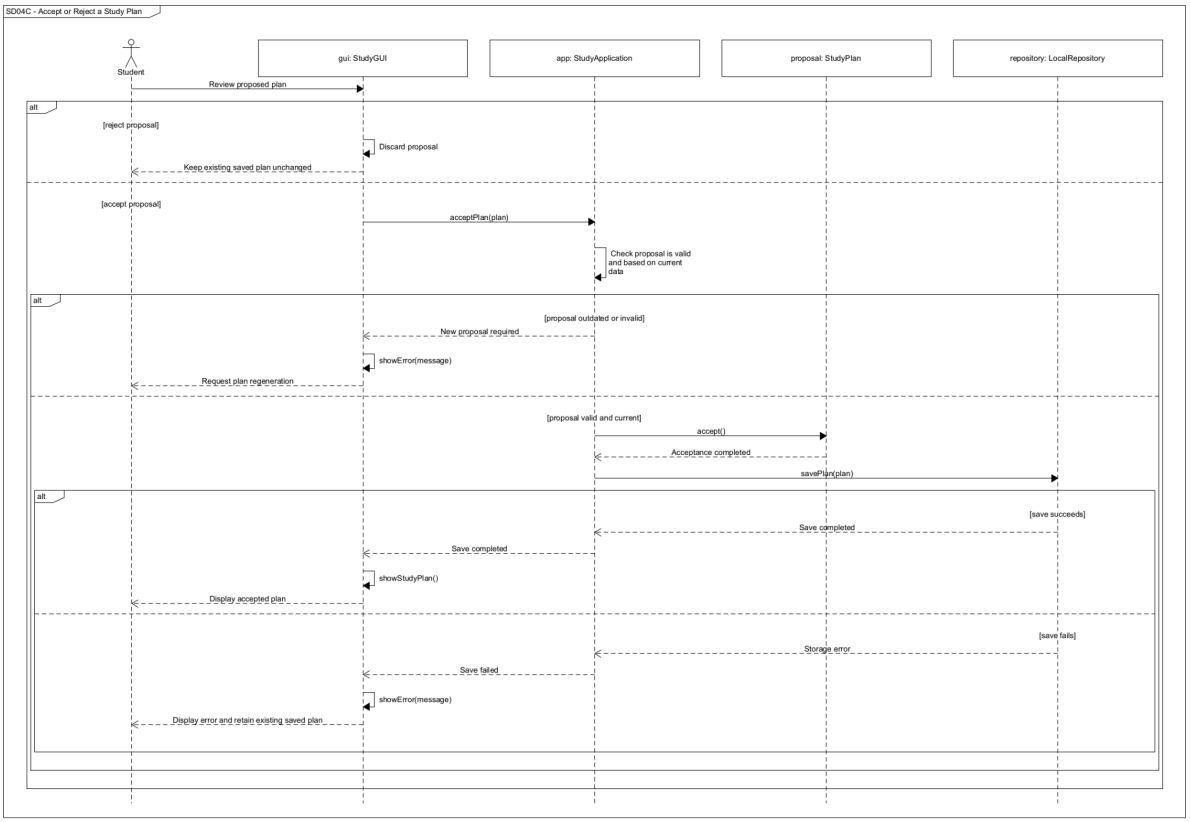
Confirm:


SD03B - Confirm Syllabus Tasks
Student
gui: StudyGUI
app: StudyApplication
syllabus: SyllabusService
repository: LocalRepository
Review drafts, complete details and select Confirm
confirmTasks(courseId, tasks)
confirmTasks(courseId, tasks)
Validate selected tasks and required details
alt
[details invalid or incomplete]
Validation error
Validation error
showError(message)
Request corrections
[details valid]
loop
[for each confirmed task]
saveTask(task)
break
[save fails]
Storage error
Confirmation failed
Confirmation failed
showError(message)
Display saving error
Task saved
Confirmation completed
Confirmation completed
showCoursesAndTasks()
Display imported tasks

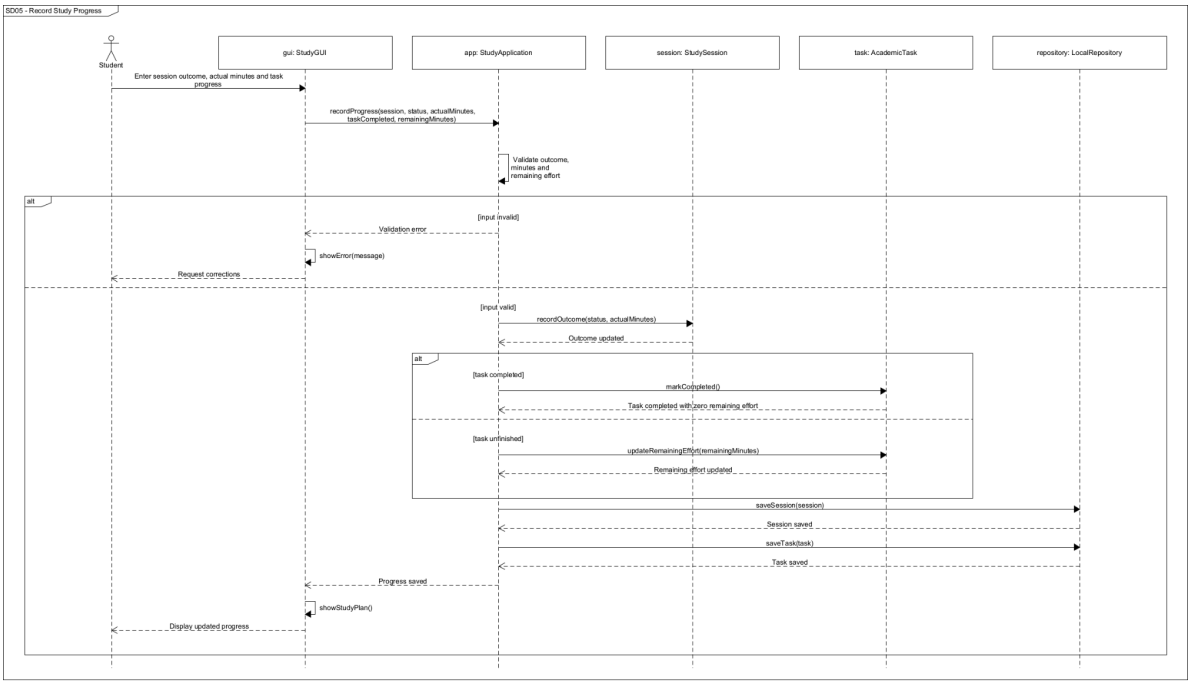
# SD04A - Generate or Revise a Study Plan



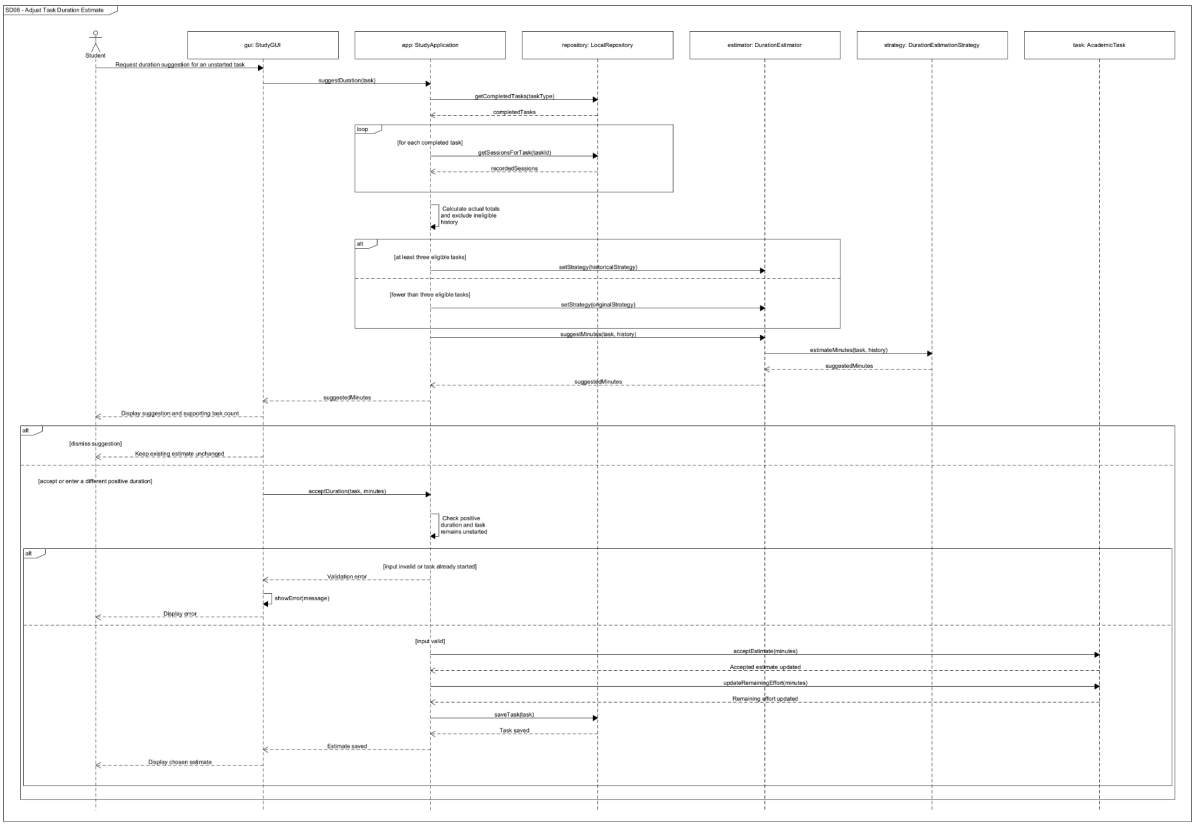

SD04B - Scheduling Tool Execution
agent: StudyAgent
tools: ToolManager
scheduling: SchedulingTool
repository: LocalRepository
scheduler: Scheduler
executeTool(name, arguments)
Check registered tool name and arguments
alt
[invalid tool request]
toolError
[valid scheduling request]
execute(arguments)
loop
[for each relevant course]
getTasks(courseId)
courseTasks
getPeriods()
periods
getPlans(weekStart)
existingPlans
Select unfinished tasks and preserve past sessions for revision
getAvailableSlots(weekStart, periods)
slots
Exclude past slots and slots conflicting with preserved sessions
createSchedule(weekStart, orderedTasks, slots)
proposedPlan
Include preserved past sessions when revising
validatePlan(proposedPlan, tasks, periods)
valid
alt
[plan valid]
Validated proposal and unscheduled effort
toolResult
[plan invalid]
Validation errors
toolError



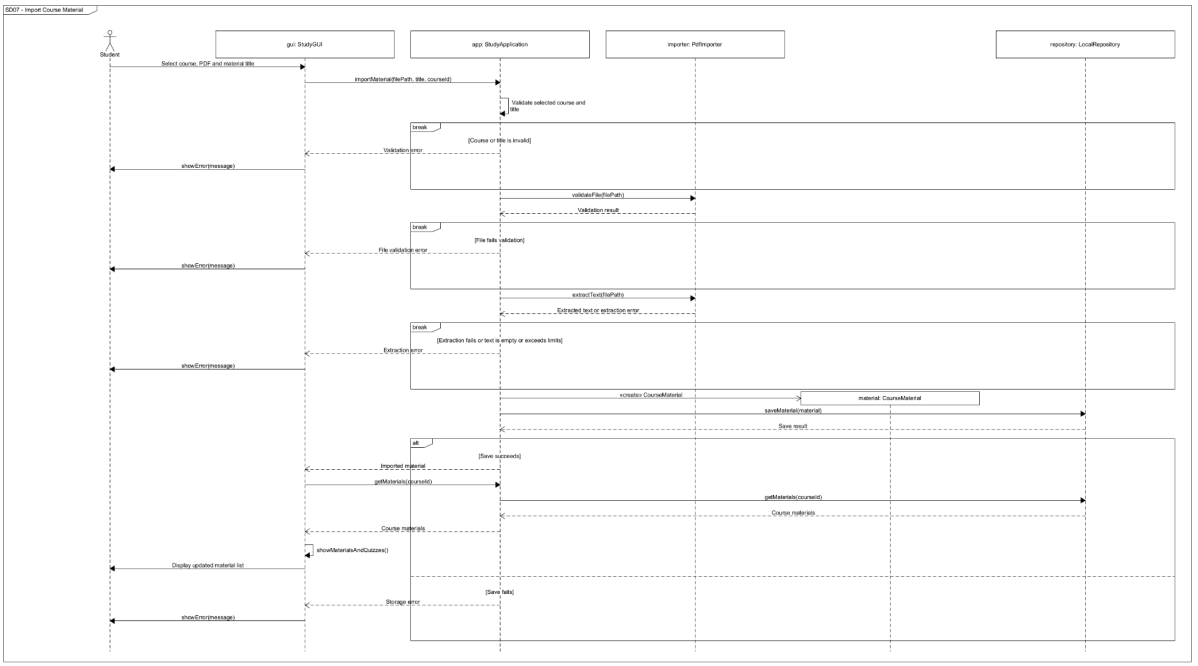

# SD05 - Record Study Progress



# SD06 - Adjust Task Duration Estimate

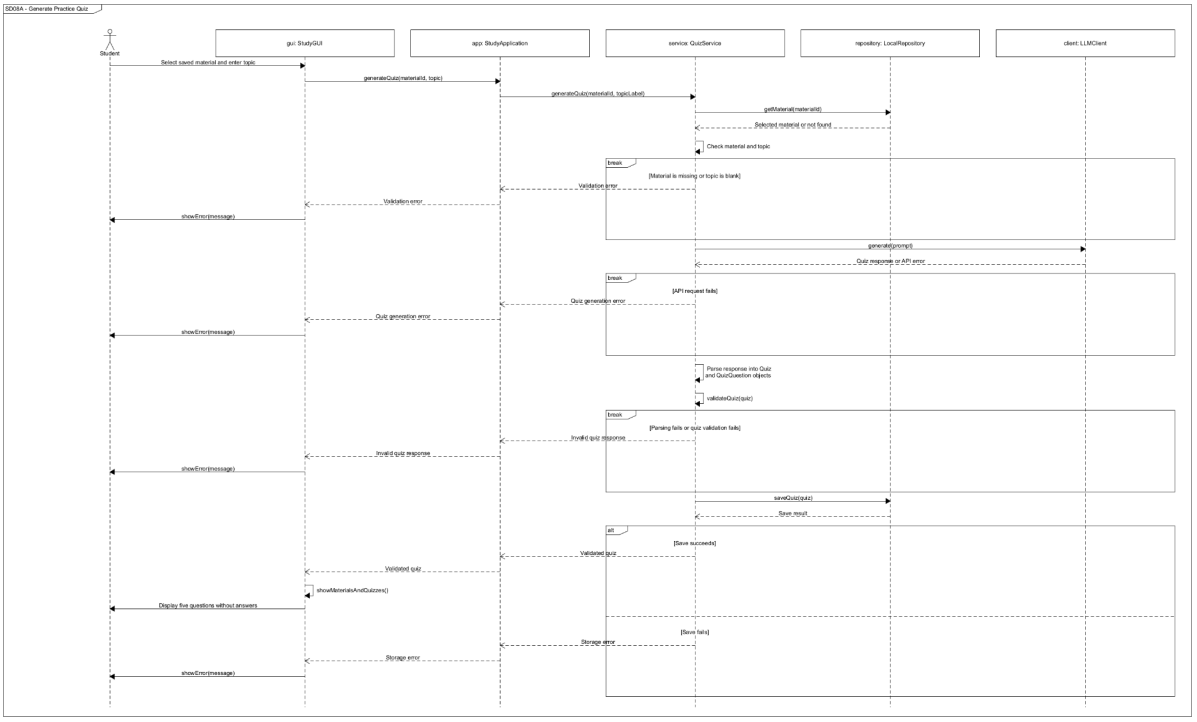



# SD07 - Import Course Material

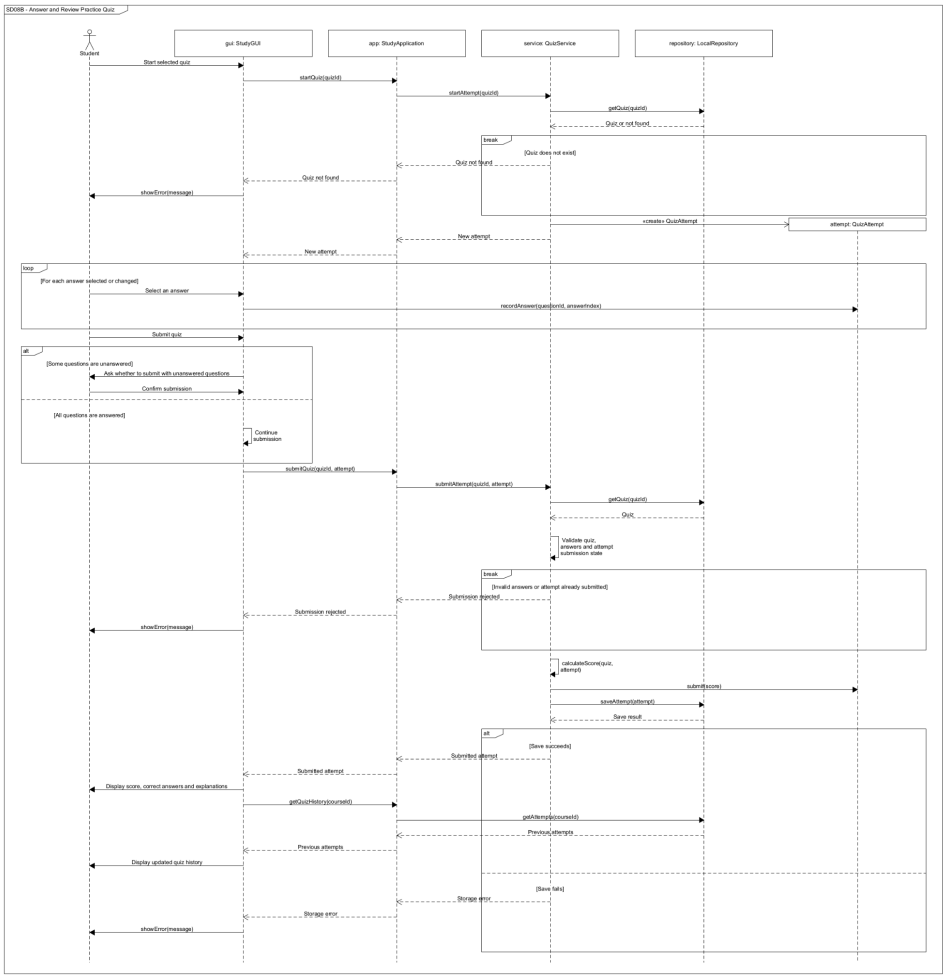


# SD08A - Generate Practice

Generate:

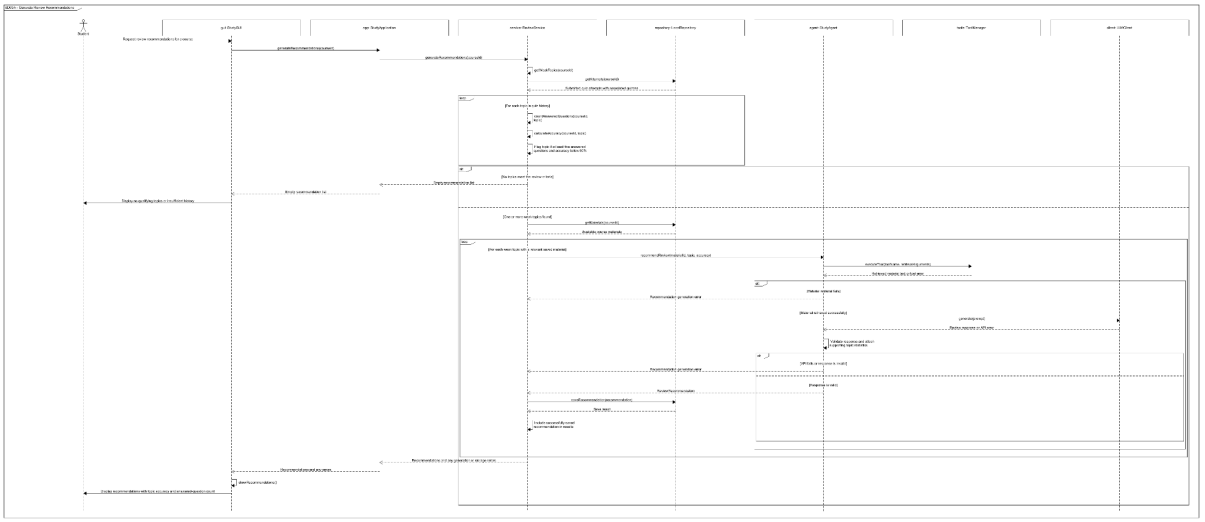



Answer & Review:

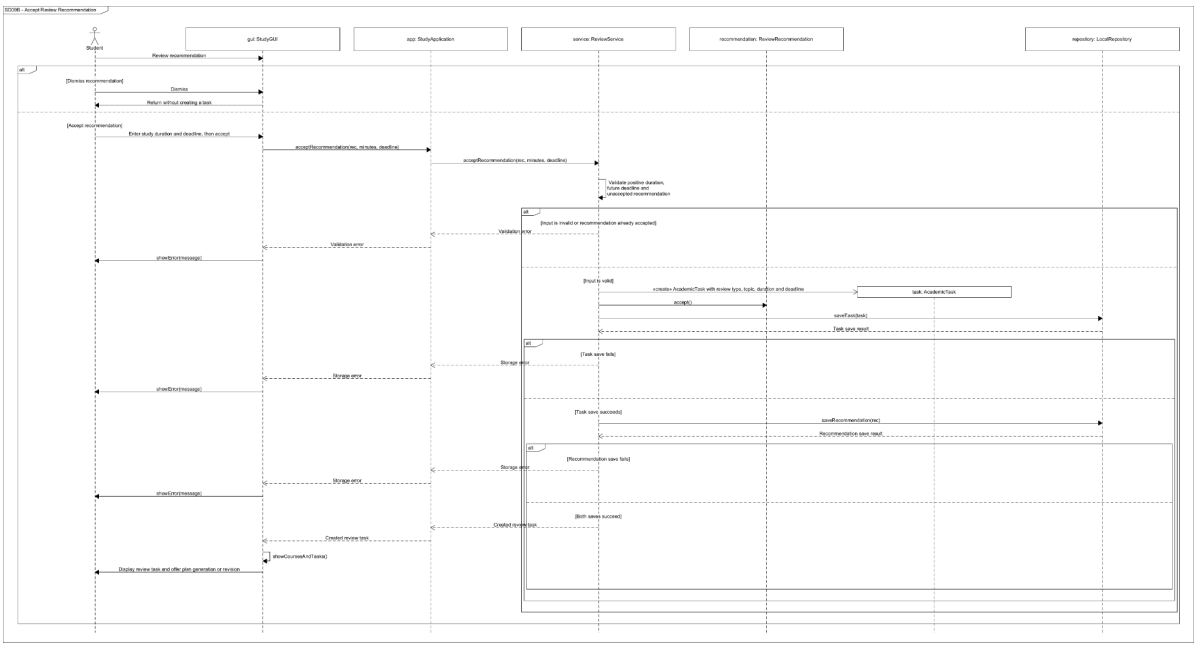


# SD09A - Generate Review Recommendations

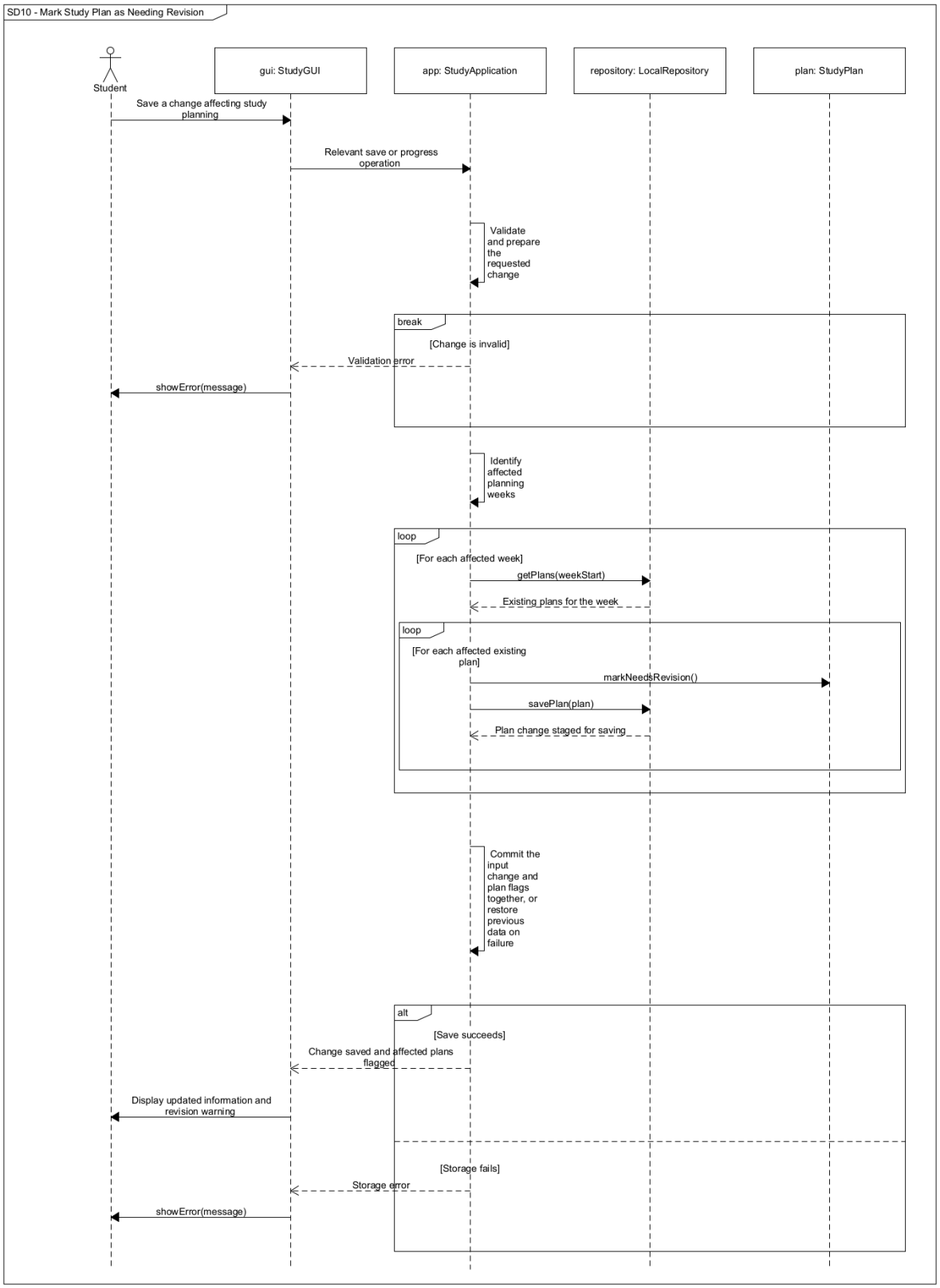
Generate:



Accept Review Recommendation:



## SD10 - Mark Study Plan as Needing Revision




This shared interaction supplements SD01, SD02, SD03B, SD05, SD06, and SD09B. The initiating operation is saveTask(), savePeriod(), removePeriod(), confirmTasks(),

recordProgress(), acceptDuration(), or acceptRecommendation(), as applicable. Existing sessions remain unchanged until the student accepts a revised plan through SD04. Task removal additionally cancels associated future sessions, as shown in SD01B.

# Task 3:

## Feature-to-Design Traceability

**Shared components:** Every feature uses **StudyApplication** as its application entry point and **LocalRepository** for persistence. **StudyGUI** and **StudyCLI** provide access to the shared application logic through **MVC** and **Facade**. The classes column lists additional feature-specific participants. Sequence diagrams illustrate the GUI path.

| Feature | Type | Related use case | Additional participating classes/interfaces | Key methods | Sequence diagrams | Design patterns |
|---|---|---|---|---|---|---|
| **F01 - Course and Academic Task Management** | Deterministic | **UC01 - Manage Courses and Tasks** | Course, AcademicTask, StudyPlan | saveCourse(), removeCourse(), saveTask(), removeTask(), archive(), markNeedsRevision() | SD01, SD01B, SD01C, SD10 | MVC, Facade |
| **F02 - Weekly Availability and Commitment Management** | Deterministic | **UC02 - Manage Weekly Availability** | WeeklyPeriod, StudyPlan | savePeriod(), removePeriod(), getPeriods(), markNeedsRevision() | SD02, SD10 | MVC, Facade |
| **F03 - Syllabus Import and Deadline Extraction** | Hybrid | **UC03 - Import Syllabus Tasks** | SyllabusService, PdfImporter, LLMClient, DeepSeekClient, AcademicTask, StudyPlan | importSyllabus(), extractTasks(), validateFile(), extractText(), generate(), validateResponse(), confirmTasks(), markNeedsRevision() | SD03A, SD03B, SD10 | MVC, Facade, Adapter |

| | | | | | | |
|---|---|---|---|---|---|---|
| **F04 - Weekly Study Plan Generation** | Hybrid | **UC04 - Generate Weekly Study Plan** | StudyAgent, ToolManager, AgentTool, StudyDataTool, SchedulingTool, Scheduler, TimeSlot, StudyPlan, StudySession, AcademicTask, WeeklyPeriod, LLMClient, DeepSeekClient | generatePlan(), getToolDescriptions(), executeTool(), execute(), generate(), getAvailableSlots(), createSchedule(), validatePlan(), acceptPlan(), savePlan() | SD04A, SD04B, SD04C | MVC, Facade, Command, Adapter |
| **F05 - Study Plan Revision** | Hybrid | **UC05 - Revise Study Plan** | StudyAgent, ToolManager, AgentTool, StudyDataTool, SchedulingTool, Scheduler, TimeSlot, StudyPlan, StudySession, AcademicTask, WeeklyPeriod, LLMClient, DeepSeekClient | revisePlan(), executeTool(), execute(), generate(), getAvailableSlots(), createSchedule(), validatePlan(), acceptPlan(), savePlan() | SD04A, SD04B, SD04C | MVC, Facade, Command, Adapter |
| **F06 - Study Session and Task Progress Recording** | Deterministic | **UC06 - Record Study Progress** | StudySession, AcademicTask, StudyPlan | recordProgress(), recordOutcome(), markCompleted(), updateRemainingEffort(), saveSession(), saveTask(), markNeedsRevision() | SD05, SD10 | MVC, Facade |
| **F07 - Adaptive Task Duration Estimation** | Deterministic | **UC07 - Adjust Task Duration Estimate** | DurationEstimator, DurationEstimationStrategy, OriginalDurationStrategy, HistoricalDurationStrategy, AcademicTask, StudySession, StudyPlan | suggestDuration(), getCompletedTasks(), getSessionsForTask(), setStrategy(), suggestMinutes(), estimateMinutes(), acceptDuration(), acceptEstimate(), markNeedsRevision() | SD06, SD10 | MVC, Facade, Strategy |

| | | | | | | |
|---|---|---|---|---|---|---|
| **F08 - Course Material Import and Retrieval** | Deterministic | **UC08 - Import and Retrieve Course Materials** | PdfImporter, CourseMaterial, QuizService, StudyAgent, ToolManager, AgentTool, StudyDataTool | importMaterial(), validateFile(), extractText(), saveMaterial(), getMaterials(), getMaterial(), executeTool(), execute() | SD07; retrieval portions of SD08A and SD09A | MVC, Facade; Command for agent retrieval |
| **F09 - Practice Quiz Generation and Assessment** | Hybrid | **UC09 - Take and Review Practice Quiz** | QuizService, Quiz, QuizQuestion, QuizAttempt, CourseMaterial, LLMClient, DeepSeekClient | generateQuiz(), generate(), validateQuiz(), startQuiz(), startAttempt(), recordAnswer(), submitQuiz(), submitAttempt(), calculateScore(), submit(), saveAttempt(), getQuizHistory() | SD08A, SD08B | MVC, Facade, Adapter |
| **F10 - Weak-Topic Review Recommendations** | Hybrid | **UC10 - Get and Accept Review Recommendations** | ReviewService, StudyAgent, ToolManager, AgentTool, StudyDataTool, QuizAttempt, Quiz, QuizQuestion, CourseMaterial, ReviewRecommendation, AcademicTask, StudyPlan, LLMClient, DeepSeekClient | generateRecommendations(), getWeakTopics(), countAnsweredQuestions(), calculateAccuracy(), recommendReview(), executeTool(), generate(), saveRecommendation(), acceptRecommendation(), accept(), saveTask(), markNeedsRevision() | SD09A, SD09B, SD10 | MVC, Facade, Command, Adapter |

**Cross-feature connections:** F10 creates an academic task managed through F01; scheduling that task follows F04 or F05. SD10 shows the shared process for marking affected plans as needing revision.

## Task 4:

# Explain How Each Feature Is Realized

**Shared persistence behavior**: Related updates are performed within one repository transaction. Success is reported only after all changes are committed. If any update fails, staged changes are discarded and modified in-memory objects are restored.

### F01 - Course and Academic Task Management

**Related use case:** UC01 - Manage Courses and Tasks
**Related sequence diagrams:** SD01, SD01B, SD01C, SD10

**Classes and methods:** StudyGUI and StudyCLI collect user input and call StudyApplication through `saveCourse()`, `removeCourse()`, `saveTask()`, and `removeTask()`. Course and AcademicTask represent the records, LocalRepository stores them, and StudyPlan provides `markNeedsRevision()`.

**Execution:** StudyApplication validates required fields, deadlines, priorities, and positive duration estimates before saving. New records receive unique identifiers, while edits retain their existing identifiers. AcademicTask keeps the original estimate separately from accepted and remaining duration values. Changes affecting scheduling mark affected plans as needing revision.

Confirmed task removal calls `archive()` and removes associated future sessions while preserving past session history. A course can be deleted only when no associated tasks, including archived tasks, or course materials remain. Related task and plan changes are committed together so that storage failure preserves the previously saved state. The interface displays either the updated records or an error message.

**Design patterns:** MVC separates interface responsibilities from application and domain logic. Facade provides the shared StudyApplication entry point used by both interfaces.

### F02 - Weekly Availability and Commitment Management

**Related use case:** UC02 - Manage Weekly Availability
**Related sequence diagrams:** SD02, SD10

**Classes and methods:** StudyGUI and StudyCLI collect weekly periods and call StudyApplication through `savePeriod()`, `removePeriod()`, and `getPeriods()`. WeeklyPeriod represents each availability or commitment entry. LocalRepository stores the periods, Scheduler calculates usable study time through `getAvailableSlots()`, and StudyPlan provides `markNeedsRevision()`.

**Execution:** Each period contains a day of the week, start time, end time, and type. StudyApplication checks that the end time follows the start time within the same day and rejects overlapping availability entries. Valid entries are saved through LocalRepository; edits retain the existing identifier, and removal uses `deletePeriod()`.

During scheduling, Scheduler subtracts commitments from availability, giving commitments precedence wherever they overlap. Changes to weekly periods mark affected plans as needing revision while retaining their sessions until a revised plan is accepted. Related period and plan updates are saved together. Invalid input or storage failure produces an error, and the interface informs the student when no usable study time remains.

**Design patterns:** MVC separates the schedule editor from validation and scheduling logic. Facade provides consistent access to period management through StudyApplication for both GUI and CLI.

## F03 - Syllabus Import and Deadline Extraction

**Related use case:** UC03 - Import Syllabus Tasks
**Related sequence diagrams:** SD03A, SD03B, SD10

**Classes and methods:** StudyApplication's `importSyllabus()` delegates to SyllabusService's `extractTasks()`. PdfImporter provides `validateFile()` and `extractText()`. SyllabusService calls LLMClient's `generate()`, implemented by DeepSeekClient, and checks the response through `validateResponse()`. Confirmed AcademicTask objects are saved through `confirmTasks()` and LocalRepository's `saveTask()`.

**Execution:** The student selects a course and uploads a text-based PDF syllabus. PdfImporter checks the file and extracts its text within configured size and text-length limits. SyllabusService sends the text to the model to identify proposed tasks and deadlines, validates the response, and returns editable drafts.

The student corrects ambiguous dates and supplies missing details, including task type, priority, and duration, before confirming which tasks to save. Confirmation validates the completed drafts and persists them. Affected plans are marked as needing revision through StudyPlan's `markNeedsRevision()`. Cancelling before confirmation saves no tasks.

Unsupported files, extraction failures, empty text, invalid model output, or API errors produce an explanation and allow manual task entry. If saving is interrupted, retries retain the draft identifiers to avoid duplicating tasks already saved.

**Design patterns:** Adapter allows DeepSeekClient to expose the common LLMClient interface. Facade coordinates the workflow through StudyApplication, while MVC separates user interaction from extraction, validation, and persistence.

## F04 - Weekly Study Plan Generation

**Related use case:** UC04 - Generate Weekly Study Plan
**Related sequence diagrams:** SD04A, SD04B, SD04C

**Classes and methods:** StudyApplication calls StudyAgent's `generatePlan()`. The agent uses LLMClient's `generate()` and ToolManager's `executeTool()` to access StudyDataTool and SchedulingTool. Scheduler provides `getAvailableSlots()`, `createSchedule()`, and `validatePlan()`. StudyPlan holds the proposed StudySession objects, priority explanation, and unscheduled effort. Acceptance uses StudyApplication's `acceptPlan()` and LocalRepository's `savePlan()`.

**Execution:** The student selects a week and requests a plan. StudyAgent retrieves tasks, availability, commitments, and relevant history through its tools. The model proposes task priorities, while SchedulingTool delegates session allocation to Scheduler. Scheduling uses remaining task effort and accepted estimates, excludes commitments, and checks that sessions fit available periods, avoid overlaps, and finish before task deadlines.

The agent's interaction loop stops when a valid proposal is ready, an error occurs, or the configured step limit is reached. The student sees the proposed sessions, priority explanation, and any unscheduled work. Acceptance rechecks the proposal against current data and saves it; rejection leaves stored plans unchanged.

Missing availability or required duration estimates prevents generation. Insufficient usable time produces a partial plan with unscheduled effort identified. API failure, invalid model output, or failed schedule validation prevents an invalid proposal from being saved.

**Design patterns:** Command provides a common AgentTool interface for data retrieval and scheduling operations. Adapter connects DeepSeekClient through LLMClient. Facade coordinates access through StudyApplication, while MVC separates the interfaces from planning logic.

## F05 - Study Plan Revision

**Related use case:** UC05 - Revise Study Plan
**Related sequence diagrams:** SD04A, SD04B, SD04C

**Classes and methods:** StudyApplication delegates revision to StudyAgent's `revisePlan()`. ToolManager's `executeTool()` coordinates StudyDataTool and SchedulingTool. Scheduler uses `getAvailableSlots()`, `createSchedule()`, and `validatePlan()` to rebuild future sessions. StudyPlan and StudySession represent the proposal, while `acceptPlan()` and LocalRepository's `savePlan()` handle acceptance and persistence.

**Execution:** The student requests a revision for an existing weekly plan after changes to workload, availability, or progress. StudyAgent retrieves the current constraints and unfinished tasks, reassesses priorities through LLMClient, and invokes SchedulingTool.

Scheduler rebuilds future sessions using remaining task effort while preserving past session records and recorded progress.

The proposal includes an explanation of significant changes and identifies any work that cannot fit. Validation checks the revised future sessions against current availability, commitments, and deadlines. Before acceptance, StudyApplication checks that the proposal still reflects the latest data.

Acceptance saves the revision under the existing plan identifier, replaces its future sessions, and clears the revision warning. The update is committed together so a storage failure preserves the previous plan. Rejection, API failure, or invalid output leaves the current plan and any existing revision warning unchanged.

**Design patterns:** Command supports reusable retrieval and scheduling tools. Adapter isolates the DeepSeek integration behind LLMClient. Facade provides the revision entry point through StudyApplication, and MVC separates presentation from revision logic.

## F06 - Study Session and Task Progress Recording

**Related use case:** UC06 - Record Study Progress
**Related sequence diagrams:** SD05, SD10

**Classes and methods:** StudyGUI and StudyCLI call StudyApplication's `recordProgress()`. StudySession records the outcome through `recordOutcome()`, while AcademicTask provides `updateRemainingEffort()` and `markCompleted()`. LocalRepository stores changes through `saveSession()` and `saveTask()`, and StudyPlan provides `markNeedsRevision()`.

**Execution:** The student selects a session and records whether it was completed, partially completed, or missed, together with actual study minutes. The student separately confirms whether the academic task is finished and supplies an estimated remaining duration for unfinished work. StudyApplication rejects negative durations and requires missed sessions to record zero study minutes.

The session outcome and task progress are updated together. Completing a session does not automatically complete its task; `markCompleted()` is called only when the student confirms task completion, setting remaining effort to zero. Otherwise, `updateRemainingEffort()` stores the student's remaining-effort estimate.

Editing a session retains its identifier and replaces the previous outcome. Actual study time is totalled from saved session records, preventing duplicate counting. Changes affecting future sessions mark the relevant plans as needing revision. Session, task, and plan updates are committed together, with storage failure preserving the previously saved state.

**Design patterns:** MVC separates progress entry from validation and domain updates. Facade provides the shared `recordProgress()` operation through StudyApplication for both interfaces.

## F07 - Adaptive Task Duration Estimation

**Related use case:** UC07 - Adjust Task Duration Estimate
**Related sequence diagrams:** SD06, SD10

**Classes and methods:** StudyApplication's `suggestDuration()` retrieves history through LocalRepository's `getCompletedTasks()` and `getSessionsForTask()`. DurationEstimator uses `setStrategy()` and `suggestMinutes()` to delegate to DurationEstimationStrategy's `estimateMinutes()`. OriginalDurationStrategy and HistoricalDurationStrategy provide the alternative calculations. Acceptance uses `acceptDuration()` and AcademicTask's `acceptEstimate()`.

**Execution:** When creating or editing an unstarted task, the student requests an adjusted estimate. The application retrieves completed tasks of the same type and totals their recorded actual study minutes. Records with missing or zero original estimates or actual durations are excluded.

With at least three eligible historical tasks, HistoricalDurationStrategy calculates each task's actual-to-original-estimated duration ratio, averages the ratios, and applies the average to the current task's original estimate. The result is rounded to a positive whole number of minutes. With fewer than three eligible tasks, OriginalDurationStrategy returns the original estimate. The interface displays the suggestion and supporting task count.

The student may accept, override, or dismiss the suggestion. Acceptance requires a positive duration and an unstarted task. It updates accepted and remaining minutes while preserving the original estimate. LocalRepository saves the change, and affected plans are marked as needing revision. Dismissing the suggestion leaves the stored estimate unchanged.

**Design patterns:** Strategy separates historical adjustment from the original-estimate fallback. Facade coordinates the operation through StudyApplication, while MVC separates the interfaces from estimation logic.

## F08 - Course Material Import and Retrieval

**Related use case:** UC08 - Import and Retrieve Course Materials
**Related sequence diagrams:** SD07; retrieval portions of SD08A and SD09A

**Classes and methods:** StudyApplication provides `importMaterial()` and `getMaterials()`. PdfImporter checks and reads uploads through `validateFile()` and `extractText()`. CourseMaterial stores the title and extracted text, while LocalRepository provides `saveMaterial()`, `getMaterials()`, and `getMaterial()`. StudyDataTool supports agent retrieval through ToolManager's `executeTool()`.

**Execution:** The student selects a course, enters a material title, and uploads a text-based PDF. StudyApplication validates the course and title. PdfImporter checks the file type and configured size limit, extracts the text, and checks that it is non-empty and within the

text-length limit. Valid content is saved as a CourseMaterial record associated with the selected course.

The interface lists saved materials through `getMaterials(courseId)`. When the student selects a material for a quiz, QuizService retrieves it through `getMaterial(materialId)`. For review recommendations, StudyAgent retrieves the associated material through StudyDataTool. Retrieval uses saved identifiers and explicit material associations rather than semantic search.

Unsupported, oversized, corrupted, password-protected, scanned, or text-empty files are rejected with an explanation. Materials exceeding the text limit require a shorter PDF. Failed imports leave existing materials unchanged, and missing records produce a retrieval error.

**Design patterns:** Facade provides the shared import and listing operations through StudyApplication. MVC separates interface behavior from file processing and storage. Command exposes agent retrieval through the common AgentTool interface.

## F09 - Practice Quiz Generation and Assessment

**Related use case:** UC09 - Take and Review Practice Quiz
**Related sequence diagrams:** SD08A, SD08B

**Classes and methods:** StudyApplication provides `generateQuiz()`, `startQuiz()`, `submitQuiz()`, and `getQuizHistory()`. QuizService coordinates generation and assessment through `validateQuiz()`, `startAttempt()`, `submitAttempt()`, and `calculateScore()`. LLMClient's `generate()` supplies the model response. Quiz, QuizQuestion, and QuizAttempt represent the questions and answers, while LocalRepository stores quizzes and attempts.

**Execution:** The student selects saved material and enters one topic label. QuizService retrieves the material and requests five multiple-choice questions from the model. It validates that the response contains exactly five questions, each with four choices, one designated correct answer, and an explanation. A valid quiz is saved with its course, material, and topic associations.

Starting the quiz creates a QuizAttempt. Answer selections are recorded through `recordAnswer()`. On submission, unanswered questions prompt confirmation and receive no credit if submitted. QuizService validates the answers and calculates the score deterministically using the saved answer key. The submitted attempt is saved before the interface displays the score, correct answers, and explanations.

Each attempt can be submitted only once. Saved results are grouped by course and topic for history and review recommendations. Missing material, insufficient relevant content, malformed output, or API failure prevents quiz creation. Storage failures produce an error without reporting a successful submission. Structural validation checks the quiz format but cannot guarantee factual accuracy.

**Design patterns:** Adapter connects DeepSeekClient through LLMClient. Facade provides the shared quiz operations through StudyApplication, while MVC separates quiz presentation from generation, scoring, and persistence

## F10 - Weak-Topic Review Recommendations

**Related use case:** UC10 - Get and Accept Review Recommendations
**Related sequence diagrams:** SD09A, SD09B, SD10

**Classes and methods:** StudyApplication delegates to ReviewService's `generateRecommendations()` and `acceptRecommendation()`. ReviewService uses `countAnsweredQuestions()`, `calculateAccuracy()`, and `getWeakTopics()`. StudyAgent's `recommendReview()` retrieves material through ToolManager and StudyDataTool, then calls LLMClient's `generate()`. ReviewRecommendation stores the suggestion and supporting statistics, while AcademicTask represents an accepted review task.

**Execution:** ReviewService groups submitted quiz results by course and saved topic label. Topic accuracy is calculated as correct answered questions divided by total answered questions, expressed as a percentage. Topics below 60% accuracy with at least five answered questions qualify for recommendations. Topics with fewer answers are labelled as having insufficient evidence.

For each qualifying topic, StudyAgent retrieves the associated course material through a tool and requests a focused review activity from the model. Valid recommendations are saved through LocalRepository and displayed with their supporting accuracy and answered-question count. Missing material, invalid output, or API failure produces an explanation while preserving quiz history and calculated scores.

To accept a recommendation, the student supplies a positive duration and a future deadline. ReviewService checks that it has not already been accepted, creates a review AcademicTask, and marks the recommendation accepted. These changes and any affected plan flags are saved together to prevent partial acceptance or duplicate tasks. The student can then schedule the task through F04 or F05; acceptance alone does not allocate study sessions.

**Design patterns:** Command supports material retrieval through AgentTool. Adapter connects the model through LLMClient. Facade coordinates access through StudyApplication, while MVC separates recommendation presentation from analysis and task creation.